

Inheritance

Inheritance is one of the most important concepts in object-oriented programming along with encapsulation and polymorphism. Basically it allows code reuse because a sub-class (child or derived class) can easily use the properties and methods defined in the super class (parent class).

The goal of this lab is to expose the concept of inheritance in Java. It will show how to create a Java class by extending another class. It will also be seen that the class Object is the mother of all classes. You will learn to override a method of the parent class. We will see how to type casting between instances of objects that are related through inheritance. Finally we will see the use of the final classes and methods.

Means

- **Inheritance** of Java Tutorial
- **Overriding vs Hiding** article from javaranch.com

Training

- **Exercise 1: Compile and run a Java program that uses classes that are related through inheritance**
- **Exercise 2: Constructor Call String and the Super Keyword**
- **Exercise 3: Method Overriding**
- **Exercise 4: Type casting**
- **Exercise 5: Classes and final methods**
- **Exercise 6: Building an application**
- **Homework**

Exercise 1: Compile and run a Java program that uses classes that are related through inheritance

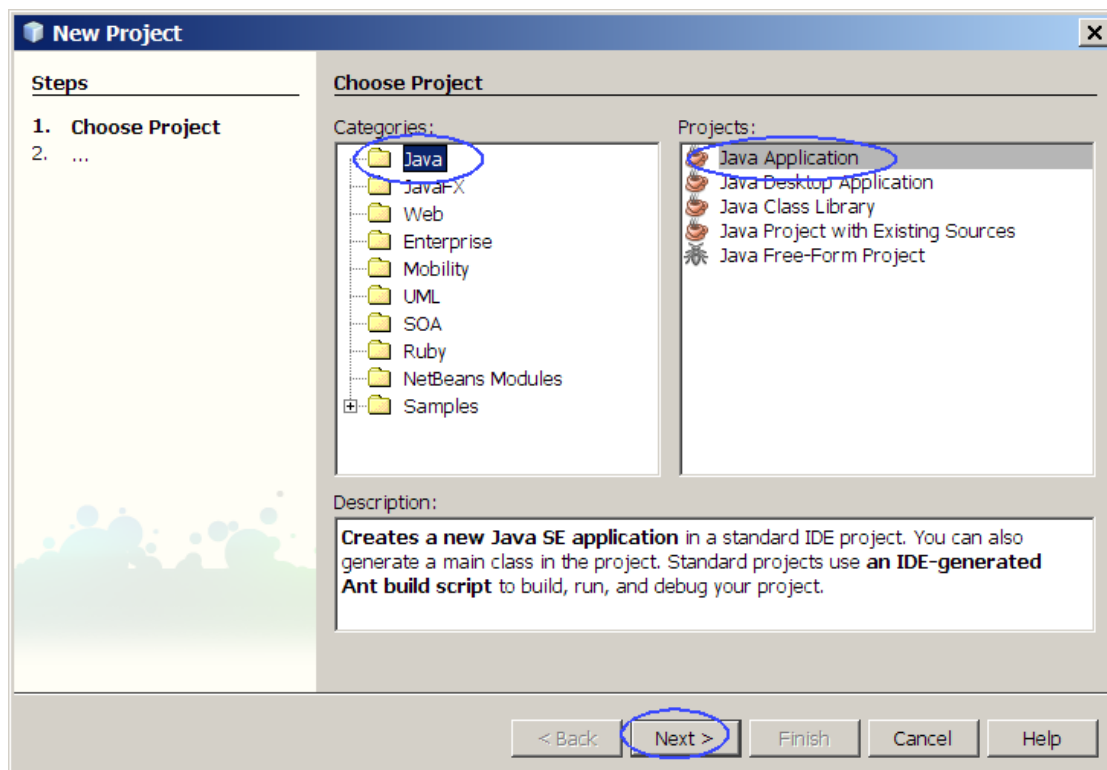
In this exercise you are going to write a program in which several classes, which are related through inheritance, are created and used. First a Person class is created. Subclasses of the Person class, Student class and Teacher class are then created. A subclass of the Student class is also created, which is called the InternationalStudent class.

1. **Compile and run a Java program that uses related classes**
2. **View the inheritance hierarchy with the IDE**

(1.1) Compile and run a Java program that uses related classes

1. Create a new NetBeans project

- Select **File -> New Project (Ctrl + Shift + N)** . The **New Project** dialog appears.
- In the **Choose Project** section select **Java** in the **Categories** section and **Java Application** in the **Projects** section .
- Click on **Next** .



- In the **Name and Location** section , for the **Project Name** field , type **MyPeopleExample** . (Figure-1.10)
- Click **Finish** .

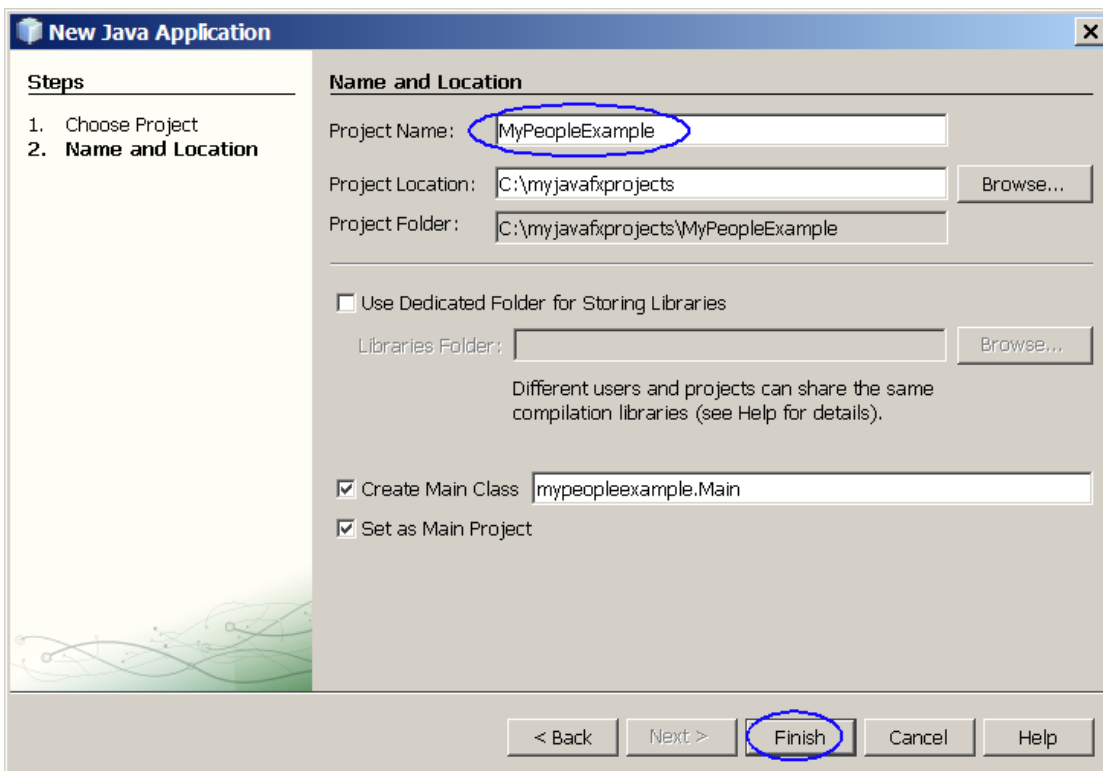
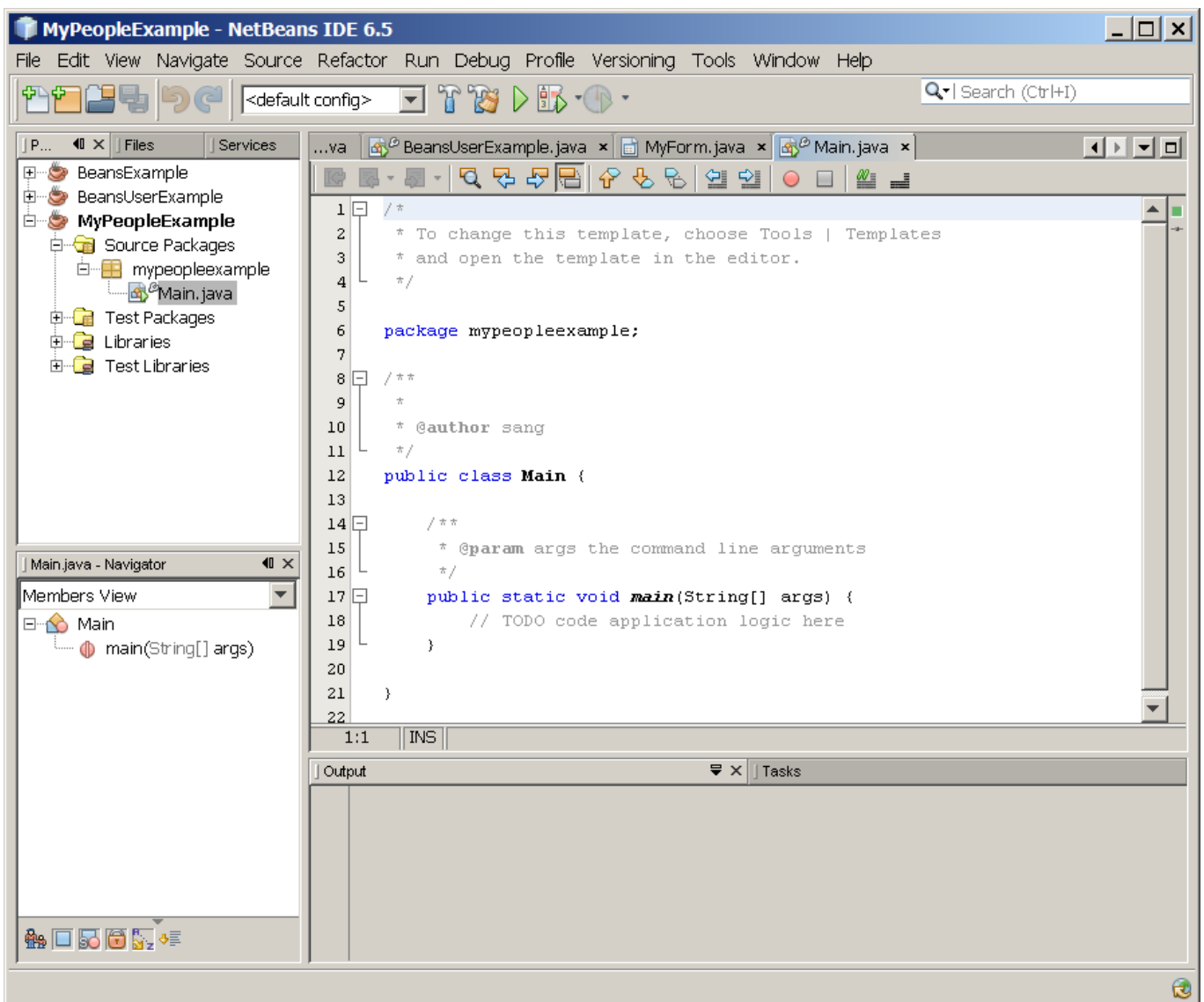


Figure-1.10: Create a new project

- Notice that the project node **MyPeopleExample** is created in the **Projects** section of the NetBeans IDE and that **Main.java** is displayed in the IDE editor window.



2. Write **Person.java** .

- Right click on the **MyPeopleExample** project and select **New-> Java Class** .
- Notice that the **New Java Class** dialog box appears with the **Name and Location** section .
- In the **Class Name** field write **Person** .
 - In the **Package** field , type **mypeopleexample** or select **mypeopleexample** from the drop-down menu. (Figure-1.11)
- Click on **Finish** .

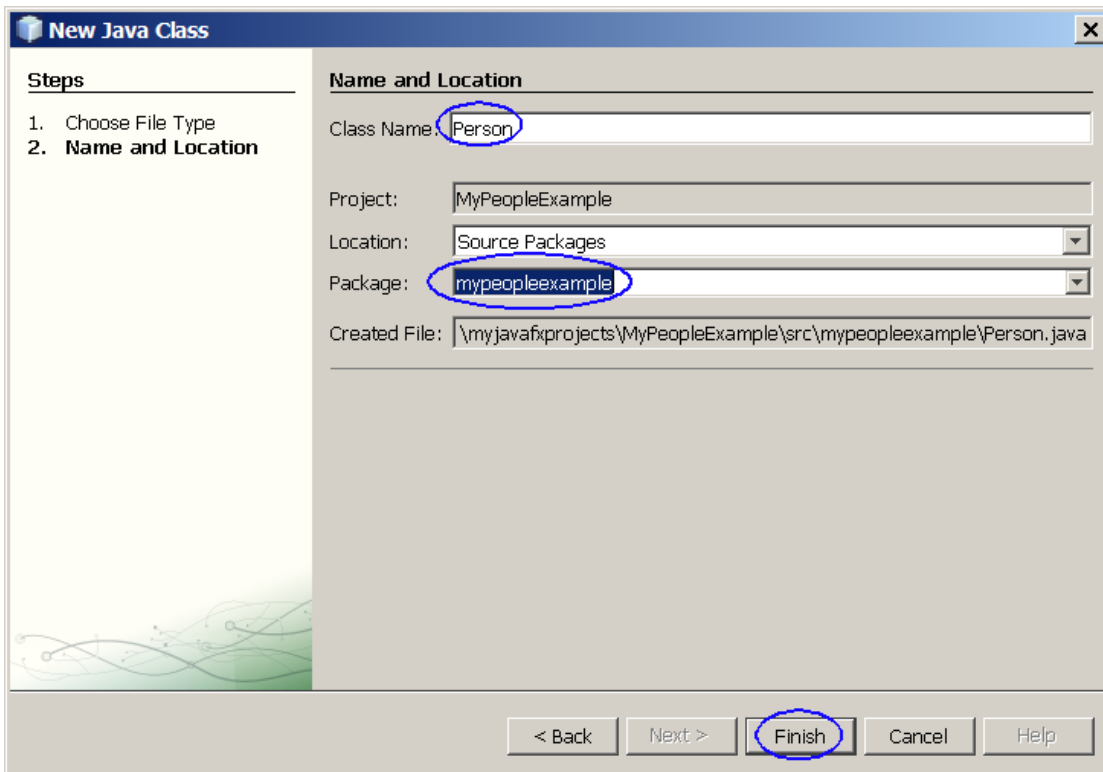


Figure-1.11: Creation of Person.java

- Observe that it is generated and displayed in the editor window **Person.java** .
- Modificar **Person.java** como se muestra en Código-1.12. Notar que la clase **Person** tiene dos campos, **name** y **address**, y los métodos getter y setter para ellos.

```
package mypeopleexample;

public class Person {

    private String name;
    private String address;

    public String getName () {
        return name;
    }

    public void setName (String name) {
        this.name = name;
    }

    public String getAddress () {
        return address;
    }

    public void setAddress (String address) {
        this.address = address;
    }

}
```

Code-1.12: Person.java

3. Write **Student.java** . Notice that the **Student** class is a subclass of the **Person** class .

- Right click on the **MyPeopleExample** project and select **New-> Java Class** .
- Notice that the **New Java Class** dialog box appears with the **Name and Location** section .
- In the **Class Name** field write **Student** .
- In the **Package** field , type **mypeopleexample** or select **mypeopleexample** from the drop-down menu. (Figure-1.11)
- Click on **Finish** .

New Java Class

Steps

1. Choose File Type
2. **Name and Location**

Name and Location

Class Name:

Project:

Location:

Package:

Created File:

< Back Next > **Finish** Cancel Help

- Observe that it is generated and displayed in the **Student.java** editor window .
- Modify **Student.java** as shown in Code-1.13. Notice that the **Student** class has two fields, **school** and **grade** , and getter and setter methods for them.

```
package mypeopleexample;

public class Student extends Person {

    private String school;
    private double grade;

    public String getSchool () {
        return school;
    }

    public void setSchool (String school) {
        this.school = school;
    }

    public double getGrade () {
        return grade;
    }

    public void setGrade (double grade) {
        this.grade = grade;
    }
}
```

Code-1.13: Student.java

4. Write **InternationalStudent.java** . Notice that the **InternationalStudent** class is a subclass of the **Student** class .

- Right click on the **MyPeopleExample** project and select **New-> Java Class** .
- Notice that the **New Java Class** dialog box appears with the **Name and Location** section .
- In the **Class Name** field write **InternationalStudent** .
- In the **Package** field , type **mypeopleexample** or select **mypeopleexample** from the drop-down menu.
- Click on **Finish** .

New Java Class

Steps

1. Choose File Type
2. Name and Location

Name and Location

Class Name:

Project:

Location:

Package:

Created File:

< Back Next > **Finish** Cancel Help

- Observe that it is generated and displayed in the **InternationalStudent.java** editor window .
- Modify **International Student.java** as shown in Code-1.14. Note that the **InternationalStudent** class has a field, **country** , and getter and setter methods.

```
package mypeopleexample;

public class InternationalStudent extends Student {

    private String country;

    public String getCountry () {
        return country;
    }

    public void setCountry (String country) {
        this.country = country;
    }

}
```

Code-1.14: InternationalStudent.java

5. Write **Teacher.java** . Notice that the **Teacher** class is a subclass of the **Person** class .

- Right click on the **MyPeopleExample** project and select **New-> Java Class** .
- Notice that the **New Java Class** dialog box appears with the **Name and Location** section .
- In the **Class Name** field write **Teacher** .
- In the **Package** field , type **mypeopleexample** or select **mypeopleexample** from the drop-down menu.
- Click **Finish** .

New Java Class

Steps

1. Choose File Type
2. Name and Location

Name and Location

Class Name:

Project:

Location:

Package:

Created File:

< Back Next > **Finish** Cancel Help

- Observe that it is generated and displayed in the **Teacher.java** editor window .
- Modify **Teacher.java** as shown in Code-1.16. Note that the **Teacher** class has a field, **subject** , and getter and setter methods for its management.

```
package mypeopleexample;

public class Teacher extends Person {

    private String subject;

    public String getSubject () {
        return subject;
    }

    public void setSubject (String subject) {
        this.subject = subject;
    }

}
```

Code-1.16: Teacher.java

6. Modify **Main.java** .

- Modificar the **Main.java** como se muestra en Código-1.17. El cambio es para crear objetos de las clases **Person**, **Student**, **InternationalStudent**, y **Teacher**.

```
package mypeopleexample;

public class Main {

    public static void main(String[] args) {

        // Create object instances and invoke methods.
        // Note that you can use methods defined in a parent
        // class for object instances of the child class.

        Person person1 = new Person();
        person1.setName("Tom Jones");

        Student student1 = new Student();
        student1.setName("CCR");
        student1.setSchool("Lexington High");

        InternationalStudent internationalStudent1 =
            new InternationalStudent();
        internationalStudent1.setName("Bill Clinton");
        internationalStudent1.setSchool("Lexington High");
        internationalStudent1.setCountry("Korea");

        Teacher teacher1 = new Teacher();
        teacher1.setName("Beatles");
        teacher1.setSubject("History");

        // Display name of object instances using the getName() method
        // defined in the Person class.
        System.out.println("Displaying names of all object instances...");
        System.out.println(" person1.getName() = " + person1.getName());
    }

}
```

```

        System.out.println ("student1.getName () =" + student1.getName ());
        System.out.println ("internationalStudent1.getName () =" +
            internationalStudent1.getName ());
        System.out.println ("teacher1.getName () =" + teacher1.getName ());
    }
}

```

Code-1.17: Main.java

7. Compile and run the program.

- Right click on the **MyPeopleExample** project and select **Run** .
- Observe the result in the NetBeans **Output** window . (Figure-1.18)

```

Displaying names of all object instances ...
person1.getName () = Tom Jones
student1.getName () = CCR
internationalStudent1.getName () = Bill Clinton
teacher1.getName () = Beatles

```

Figure-1.18: Result of the application execution

[Return to the beginning of the exercise](#)

(1.3) View the inheritance hierarchy through the `getSuperclass ()` method of the `Class` class

In this step, you will show the class hierarchy through the use of an API method - `getSuperclass ()` of a `Class` class - in your program.

1. Modify **Main.java** as shown in Code-1.30. The code snippet to add is in blue.

```

package mypeopleexample;

public class Main {

    public static void main(String[] args) {

        // Create object instances and invoke methods.
        // Note that you can use methods defined in a parent
        // class for object instances of the child class.

        Person person1 = new Person();
        person1.setName("Tom Jones");

        Student student1 = new Student();
        student1.setName("CCR");
        student1.setSchool("Lexington High");

        InternationalStudent internationalStudent1 =
            new InternationalStudent();
        internationalStudent1.setName("Bill Clinton");
        internationalStudent1.setSchool("Lexington High");
        internationalStudent1.setCountry("Korea");

        Teacher teacher1 = new Teacher();
        teacher1.setName("Beatles");
        teacher1.setSubject("History");

        // Display name of object instances using the getName() method
        // defined in the Person class.
        System.out.println("Displaying names of all object instances...");
        System.out.println(" person1.getName() = " + person1.getName());
        System.out.println(" student1.getName() = " + student1.getName());
        System.out.println(" internationalStudent1.getName() = " +
            internationalStudent1.getName());
        System.out.println(" teacher1.getName() = " + teacher1.getName());

        // Display the class hierarchy of the InternationalStudent
        // class through getSuperclass() method of Class class.
        Class class1 = internationalStudent1.getClass();

        System.out.println("Displaying class hierarchy of InternationalStudent Class...");
        while (class1.getSuperclass() != null){
            String child = class1.getName();
            String parent = class1.getSuperclass().getName();
            System.out.println(" " + child + " class is a child class of " + parent);
            class1 = class1.getSuperclass();
        }
    }
}

```

Código-1.30: Main.java modificado

2. Compilar y ejecutar el programa.

- Hacer click con el botón derecho en el proyecto **MyPeopleExample** y seleccionar **Run**.
- Observar el resultado en la ventana **Output** de NetBeans. (Figura-1.31)

```

Displaying names of all object instances...
person1.getName() = Tom Jones
student1.getName() = CCR

```

```
internationalStudent1.getName() = Bill Clinton
teacher1.getName() = Beatles
Displaying class hierarchy of InternationalStudent Class...
mypeopleexample.InternationalStudent class is a child class of mypeopleexample.Student
mypeopleexample.Student class is a child class of mypeopleexample.Person
mypeopleexample.Person class is a child class of java.lang.Object
```

Figura-1.31: Resultado de la ejecución de la aplicación

[Volver al inicio del ejercicio](#)

Resumen

En este ejercicio se ha visto la creación de objetos que están relacionados a través de la herencia.

[Volver al inicio](#)

Ejercicio 2: Encadenamiento de llamadas a Constructor

En este ejercicio, se experimenta con el concepto de la llamada encadenada de constructor y cómo usar el método `super()` y la referencia `super`.

1. **Cadena de llamada a Constructor**
2. **método `super()`**
3. **referencia `super`**

(2.1) Cadena de llamada a Constructor

The **MyPeopleExample** project will be used for the changes to be made in this section or another new project can be created, for example, **MyPeopleExampleConstructor**, by "copying" the **MyPeopleExample** project - Right click on the **MyPeopleExample** project and select **Copy Project**. In this part it is assumed that the new project has been created.

1. Modify **Person.java** as shown in Code-2.10. The change is to add a print statement inside the constructor. The code snippet that needs to be added is shown in blue.

```
package mypeopleexample;

public class Person {

    public Person() {
        System.out.println("Person: constructor is called");
    }

    private String name;
    private String address;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }
}
```

Código-2.10: Person.java

2. Modificar **Student.java** como se muestra en Código-2.11. El cambio consiste en añadir una sentencia de impresión dentro del constructor. El fragmento de código que se requiere añadir se muestra en azul.

```
package mypeopleexample;

public class Student extends Person {

    public Student () {
        System.out.println ("Student: constructor is called");
    }

    private String school;
    private double grade;

    public String getSchool () {
        return school;
    }

    public void setSchool (String school) {
        this.school = school;
    }

    public double getGrade () {
        return grade;
    }
}
```



```

    }

    public void setGrade (double grade) {
        this.grade = grade;
    }
}

```

Code-2.11: Student.java

3. Modify **InternationalStudent.java** as shown in Code-2.12. The change is to add a print statement inside the constructor. The code snippet that needs to be added is shown in blue.

```

package mypeopleexample;

public class InternationalStudent extends Student {

    public InternationalStudent () {
        System.out.println ("InternationalStudent: constructor is called");
    }

    private String country;

    public String getCountry () {
        return country;
    }

    public void setCountry (String country) {
        this.country = country;
    }

}

```

Code-2.12: InternationalStudent .java

4. Modify **Teacher.java** as shown in Code-2.13. The change is to add a print statement inside the constructor. The code snippet that needs to be added is shown in blue.

```

package mypeopleexample;

public class Teacher extends Person {

    public Teacher () {
        System.out.println ("Teacher: constructor is called");
    }

    private String subject;

    public String getSubject () {
        return subject;
    }

    public void setSubject (String subject) {
        this.subject = subject;
    }

}

```

Code-2.13: Teacher.java

5. Modify **Main.java** as shown in Code-2.14. The change is to add a print statement inside the constructor. The code snippet that needs to be added is shown in blue.

```

package mypeopleexample;

public class Main {

    public static void main(String[] args) {

        // Create an object instance of
        // InternationalStudent class.
        System.out.println("---- About to create an object instance of InternationalStudent class...");
        InternationalStudent internationalStudent1 =
            new InternationalStudent();

        // Create an object instance of
        // Teacher class.
        System.out.println("---- About to create an object instance of Teacher class...");
        Teacher teacher1 = new Teacher();

    }

}

```

Code-2.14: modified Main.java

6. Compile and run the program.

- Right click on the **MyPeopleExampleConstructor** project and select **Run** .
- Observe the result in the NetBeans **Output** window . (Figure-2.15)

```

---- About to create an object instance of InternationalStudent class ...
Person: constructor is called
Student: constructor is called
InternationalStudent: constructor is called

```

```
---- About to create an object instance of Teacher class ...
Person: constructor is called
Teacher: constructor is called
```

Figure-2.15: Result

[Return to the beginning of the exercise](#)

(2.2) super () method

Se puede usar el proyecto **MyPeopleExample** para el cambio que se va a realizar en este paso o se puede crear un proyecto nuevo, por ejemplo **MyPeopleExampleConstructorSuper**, mediante "copia" del proyecto **MyPeopleExample** - Hacer click con el botón derecho en el proyecto **MyPeopleExample** project y seleccionar **Copy Project**. Esta sección asume que se ha creado el proyecto nuevo.

1. Modificar **Person.java** como se muestra en Código-2.16. El cambio consiste en añadir otro constructor que se puede llamar desde una subclase. El fragmento de código que se requiere añadir se muestra en azul..

```
package mypeopleexample;

public class Person {

    public Person() {
        System.out.println("Person: constructor is called");
    }

    public Person(String name) {
        this.name = name;
        System.out.println("Person: constructor 2 is called");
    }

    private String name;
    private String address;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

}
```

Código-2.16: Person.java

2. Modificar **Student.java** como se muestra en Código-2.17. El cambio es la llamada del método constructor de la super clase usando el método **super()**. El fragmento de código que se requiere añadir se muestra en azul.

```
package mypeopleexample;

public class Student extends Person {

    public Student() {
        System.out.println("Student: constructor is called");
    }

    public Student(String name, String school, double grade) {
        super(name);
        this.school = school;
        this.grade = grade;
        System.out.println("Student: constructor 2 is called");
    }

    private String school;
    private double grade;

    public String getSchool() {
        return school;
    }

    public void setSchool(String school) {
        this.school = school;
    }

    public double getGrade() {
        return grade;
    }

    public void setGrade(double grade) {
        this.grade = grade;
    }

}
```

```
}  
}
```

Figura-2.17: Student.java

3. Modificar **InternationalStudent.java** como se muestra en Código-2.18. El cambio es la llamada del método constructor de la super clase usando el método **super()**. El fragmento de código que se requiere añadir se muestra en azul.

```
package mypeopleexample;  
  
public class InternationalStudent extends Student {  
  
    public InternationalStudent() {  
        System.out.println("InternationalStudent: constructor is called");  
    }  
  
    public InternationalStudent(String name, String school, double grade, String country) {  
        super(name, school, grade);  
        this.country = country;  
        System.out.println("InternationalStudent: constructor 2 is called");  
    }  
  
    private String country;  
  
    public String getCountry() {  
        return country;  
    }  
  
    public void setCountry(String country) {  
        this.country = country;  
    }  
  
}
```

Código-2.18: InternationalStudent.java

4. Modificar **Main.java** como se muestra en Código-2.19. El cambio consiste en crear un objeto de la clase InternationalStudent con parámetros de inicialización. El fragmento de código que se requiere añadir se muestra en azul.

```
package mypeopleexample;  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        // Create an object instance of  
        // InternationalStudent class.  
        System.out.println("---- About to create an object instance of InternationalStudent class...");  
        InternationalStudent internationalStudent1 =  
            new InternationalStudent("Sang Shin", // Name  
                                    "1 Dreamland", // Address  
                                    4.5, // Grade  
                                    "Korea"); // Country  
  
        System.out.println("internationalStudent1.getName() = " + internationalStudent1.getName());  
        System.out.println("internationalStudent1.getAddress() = " + internationalStudent1.getAddress());  
        System.out.println("internationalStudent1.getGrade() = " + internationalStudent1.getGrade());  
        System.out.println("internationalStudent1.getCountry() = " + internationalStudent1.getCountry());  
    }  
}
```

Código-2.19: Main.java

5. Compilar y ejecutar el programa.

- Hacer click con el botón derecho en el proyecto **MyPeopleExampleConstructorSuper** y seleccionar **Run**.
- Observar el resultado en la ventana **Output** de NetBeans. (Figura-2.20)

```
---- About to create an object instance of InternationalStudent class...  
Person: constructor 2 is called  
Student: constructor 2 is called  
InternationalStudent: constructor 2 is called  
internationalStudent1.getName() = Sang Shin  
internationalStudent1.getAddress() = null  
internationalStudent1.getGrade() = 4.5  
internationalStudent1.getCountry() = Korea
```

Figura-2.20: Resultado

[Volver al inicio del ejercicio](#)

(2.3) referencia super

Se puede el proyecto **MyPeopleExample** para modificarlo en esta sección o se puede crear un nuevo proyecto, por ejemplo, **MyPeopleExampleConstructorSuper2**, "copiando" el proyecto **MyPeopleExample** - hacer click con el botón derecho en el proyecto **MyPeopleExample** y seleccionar **Copy Project**. En esta sección se asume que se ha creado un nuevo proyecto.

1. Modificar **Person.java** como se muestra en Código-2.21. El cambio consiste en cambiar los modificadores de acceso de los campos a **protected** de tal manera que se puedan acceder desde las subclases. El fragmento de código que se requiere añadir se muestra en azul.

```
package mypeopleexample;
```

```

public class Person {

    public Person() {
        System.out.println("Person: constructor is called");
    }

    public Person(String name) {
        this.name = name;
        System.out.println("Person: constructor 2 is called");
    }

    protected String name;
    protected String address;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

}

```

Código-2.21: Person.java

2. Modificar **Student.java** como se muestra en Código-2.22. El cambio consiste en cambiar los modificadores de acceso de los campos a **protected** para que se puedan acceder desde las subclases. El fragmento de código que se requiere añadir se muestra en azul.

```

package mypeopleexample;

public class Student extends Person {

    public Student() {
        System.out.println("Student: constructor is called");
    }

    public Student(String name, String school, double grade) {
        super(name);
        this.school = school;
        this.grade = grade;
        System.out.println("Student: constructor 2 is called");
    }

    protected String school;
    protected double grade;

    public String getSchool() {
        return school;
    }

    public void setSchool(String school) {
        this.school = school;
    }

    public double getGrade() {
        return grade;
    }

    public void setGrade(double grade) {
        this.grade = grade;
    }

}

```

Figura-2.22: Student.java

3. Modificar **InternationalStudent.java** como se muestra en Código-2.23. El cambio consiste en cambiar los modificadores de acceso de los campos a **protected** para que se puedan acceder desde las subclases. El fragmento de código que se requiere añadir se muestra en azul.

```

package mypeopleexample;

public class InternationalStudent extends Student {

    public InternationalStudent() {
        System.out.println("InternationalStudent: constructor is called");
    }

    public InternationalStudent(String name, String school, double grade, String country) {
        super.name = name;
        super.school = school;
        super.grade = grade;
        this.country = country;
        System.out.println("InternationalStudent: constructor 2 is called");
    }

}

```

```

    }

    private String country;

    public String getCountry() {
        return country;
    }

    public void setCountry(String country) {
        this.country = country;
    }
}

```

Código-2.23: InternationalStudent.java

4. Compilar y ejecutar el programa.

- Hacer click con el botón derecho en el proyecto **MyPeopleExampleConstructorSuper2** y seleccionar **Run**.
- Observar el resultado en la ventana **Output** de NetBeans. (Figura-2.24)

```

---- About to create an object instance of InternationalStudent class...
Person: constructor is called
Student: constructor is called
InternationalStudent: constructor 2 is called
internationalStudent1.getName() = Sang Shin
internationalStudent1.getAddress() = null
internationalStudent1.getGrade() = 4.5
internationalStudent1.getCountry() = Korea

```

Figura-2.24: Resultado

[Volver al inicio del ejercicio](#)

Resumen

En este ejercicio se ha visto cómo los constructores de clases relacionadas se encadenan cuando se crea una instancia de un objeto. También se ha visto cómo usar el método `super()` para invocar un constructor de la clase padre.

[Volver al inicio](#)

Ejercicio 3: Predominancia (Overriding) de métodos

Este ejercicio muestra el concepto de overriding de métodos, que es posiblemente la característica más importante de la herencia Java.

1. **Override de métodos**
2. **Runtime-polimorfismo**
3. **Métodos ocultos** (aplica sólo a métodos estáticos)

(3.1) Override de métodos

Se puede usar el proyecto **MyPeopleExample** para los cambios que se van a hacer en esta sección o se puede crear un nuevo proyecto, por ejemplo, **MyPeopleExampleOverriding**, "copiando" el proyecto **MyPeopleExample** - Hacer click con el botón derecho en el proyecto **MyPeopleExample** y seleccionar **Copy Project**. Se asume aquí que se ha creado un nuevo proyecto.

1. Modificar **Person.java** como se muestra en Código-3.10. El cambio consiste en añadir un nuevo método **myMethod(String t)** que se priorizará por la subclase. El fragmento de código que se requiere añadir se muestra en azul. Los métodos `println` en los constructores están comentados por simplicidad.

```

package mypeopleexample;

public class Person {

    public Person () {
        // System.out.println ("Person: constructor is called");
    }

    public Person (String name) {
        this.name = name;
        // System.out.println ("Person: constructor 2 is called");
    } protected String name; protected String address; public String getName () { return name; } public void setName (String
name) { this.name = name; } public String getAddress () { return address; }

    public void setAddress (String address) {
        this.address = address;
    }
}

```

```

}

// A method that will be overridden by sub-class
public void myMethod (String t) {
    System.out.println ("myMethod (" + t + ") in Person class");
}
}

```

Code-3.10: Person.java

2. Modify **Student.java** as shown in Code-3.11. The change consists in modifying the access modifiers of the fields to **protected** so that they can be accessed from the subclass. The code snippet that needs to be added is shown in blue. The println methods in the constructors are commented out for simplicity.

```

package mypeopleexample;

public class Student extends Person { public Student () { // System.out.println ("Student: constructor is called"); } public Student
(String name, String school, double grade) { super (name); this.school = school; this.grade = grade; // System.out.println
("Student: constructor 2 is called"); } protected String school; protected double grade; public String getSchool () { return
school; } public void setSchool (String school) { this.school = school;

}

public double getGrade () {
    return grade;
}

public void setGrade (double grade) {
    this.grade = grade;
}

// A
public void overriding method myMethod (String t) {
    System.out.println ("myMethod (" + t + ") in Student class");
}
}

```

Figure-3.11: Student.java with overriding method

3. Modify **InternationalStudent.java** as shown in Code-3.12. The change consists in modifying the access modifiers of the fields to **protected** so that they can be accessed from the subclass. The code snippet that needs to be added is shown in blue. The println methods in the constructors are commented out for simplicity.

```

package mypeopleexample;

public class InternationalStudent extends Student {

    public InternationalStudent() {
        // System.out.println("InternationalStudent: constructor is called");
    }

    public InternationalStudent(String name, String school, double grade, String country) {
        super.name = name;
        super.school = school;
        super.grade = grade;
        this.country = country;
        // System.out.println("InternationalStudent: constructor 2 is called");
    }

    private String country;

    public String getCountry() {
        return country;
    }

    public void setCountry(String country) {
        this.country = country;
    }

// A overriding method

```

```

public void myMethod(String t){
    System.out.println("myMethod(" + t + ") in InternationalStudent class");
}
}

```

Código-3.12: InternationalStudent.java con método overriding

4. Modificar **Main.java** como se muestra en Código-3.13. El cambio es crear instancias de objetos de clases **Person**, **Student**, y **InternationalStudent** que están relacionados a través de la herencia e invoca el método overridden **myMethod()** para cada uno de esas instancias. El fragmento de código que se requiere añadir se muestra en azul.

```

package mypeopleexample;

public class Main {

    public static void main(String[] args) {

        System.out.println("---- Observe overriding method behavior ----");

        Person person1 = new Person();
        person1.myMethod("test1");

        Student student1 = new Student();
        student1.myMethod("test2");

        InternationalStudent internationalStudent1 =
            new InternationalStudent();
        internationalStudent1.myMethod("test3");

    }
}

```

Código-3.13: Main.java

5. Compilar y ejecutar el programa.

- Hacer click con el botón derecho en el proyecto **MyPeopleExampleOverriding** y seleccionar **Run**.
- Observar el resultado en la ventana **Output** de NetBeans. (Figura-3.14)

```

---- Observe overriding behavior ----
myMethod(test1) in Person class
myMethod(test2) in Student class
myMethod(test3) in InternationalStudent class

```

Figura-3.14: Resultado

[Volver al inicio del ejercicio](#)

(3.2) Polimorfismo en Runtime

Se puede usar el proyecto **MyPeopleExample** para los cambios que se van a hacer en esta sección o se puede crear un nuevo proyecto, por ejemplo, **MyPeopleExampleOverridingPolymorphism**, "copiando" el proyecto **MyPeopleExample** - hacer click con el botón derecho en el proyecto **MyPeopleExample** y seleccionar **Copy Project**. En lo que sigue se asume que ha creado un nuevo proyecto.

Nota: El código de ejemplo no muestra la capacidad de polimorfismo en runtime pero sí el esquema subyacente que lo permite. La capacidad del comportamiento polimórfico será explorado en otro laboratorio.

1. Modificar **Main.java** como se muestra en Código-3.20. El cambio intenta observar el comportamiento polimórfico. El fragmento de código que se requiere añadir se muestra en azul.

```

public class Main {

    public static void main(String[] args) {

        System.out.println("---- Observe overriding method behavior ----");

        Person person1 = new Person();
        person1.myMethod("test1");

        Student student1 = new Student();
        student1.myMethod("test2");

        InternationalStudent internationalStudent1 =
            new InternationalStudent();
        internationalStudent1.myMethod("test3");

        // Polymorphic behavior
        System.out.println("---- Observe polymorphic behavior ----");

        Person person2 = new Student();
        person2.myMethod("test4");

        Person person3 = new InternationalStudent();
        person3.myMethod("test5");

        Student student2 = new InternationalStudent();
        student2.myMethod("test6");

    }
}

```

Código-3.20: Main.java

2. Compilar y ejecutar el programa.

- Hacer click con el botón derecho en el proyecto **MyPeopleExampleOverridingPolymorphism** y seleccionar **Run**.
- Observar el resultado en la ventana **Output** de NetBeans. (Figura-3.21)

```
---- Observe overriding behavior ----
myMethod(test1) in Person class
myMethod(test2) in Student class
myMethod(test3) in InternationalStudent class
---- Observe polymorphic behavior ----
myMethod(test4) in Student class
myMethod(test5) in InternationalStudent class
myMethod(test6) in InternationalStudent class
```

Figura-3.21: Resultado

[Volver al inicio del ejercicio](#)

(3.3) Métodos ocultos

Se puede usar el proyecto **MyPeopleExample** para los cambios que se van a hacer en esta sección o se puede crear un nuevo proyecto, por ejemplo, **MyPeopleExampleHidingMethods**, "copiando" el proyecto **MyPeopleExample** - hacer click con el botón derecho en el proyecto **MyPeopleExample** y seleccionar **Copy Project**. En lo que sigue se asume que ha creado un nuevo proyecto.

1. Modificar **Person.java** como se muestra en Código-3.30. El cambio añade un método estático que será ocultado por la subclase. El fragmento de código que se requiere añadir se muestra en azul.

```
package mypeopleexample;

public class Person {

    public Person() {
        // System.out.println("Person: constructor is called");
    }

    public Person(String name) {
        this.name = name;
        // System.out.println("Person: constructor 2 is called");
    }

    protected String name;
    protected String address;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    // A method that will be overridden by sub-class
    public void myMethod(String t){
        System.out.println("myMethod(" + t + ") in Person class");
    }

    // A method that will be hidden by sub-class
    public static void myStaticMethod(String t){
        System.out.println("myStaticMethod(" + t + ") in Person class");
    }

}
```

Código-3.30: Person.java con un método estático

2. Modificar **Student.java** como se muestra en Código-3.31. El cambio añade un método estático que será ocultado por la subclase. El fragmento de código que se requiere añadir se muestra en azul.

```
package mypeopleexample;

public class Student extends Person {

    public Student() {
        // System.out.println("Student: constructor is called");
    }

    public Student(String name, String school, double grade) {
        super(name);
    }

}
```



```

        this.school = school;
        this.grade = grade;
        // System.out.println("Student: constructor 2 is called");
    }

    protected String school;
    protected double grade;

    public String getSchool() {
        return school;
    }

    public void setSchool(String school) {
        this.school = school;
    }

    public double getGrade() {
        return grade;
    }

    public void setGrade(double grade) {
        this.grade = grade;
    }

    // A overriding method
    public void myMethod(String t){
        System.out.println("myMethod(" + t + ") in Student class");
    }

    // A method that will be hidden by sub-class
    public static void myStaticMethod(String t){
        System.out.println("myStaticMethod(" + t + ") in Student class");
    }
}

```

Código-3.31: Student.java con un método static

3. Modificar **InternationalStudent.java** como se muestra en Código-3.32. El cambio añade un método estático que será ocultado por la subclase. El fragmento de código que se requiere añadir se muestra en azul.

```

package mypeopleexample;

public class InternationalStudent extends Student {

    public InternationalStudent() {
        // System.out.println("InternationalStudent: constructor is called");
    }

    public InternationalStudent(String name, String school, double grade, String country) {
        super.name = name;
        super.school = school;
        super.grade = grade;
        this.country = country;
        // System.out.println("InternationalStudent: constructor 2 is called");
    }

    private String country;

    public String getCountry() {
        return country;
    }

    public void setCountry(String country) {
        this.country = country;
    }

    // A overriding method
    public void myMethod(String t){
        System.out.println("myMethod(" + t + ") in InternationalStudent class");
    }

    // A method that will be hidden by sub-class
    public static void myStaticMethod(String t){
        System.out.println("myStaticMethod(" + t + ") in InternationalStudent class");
    }
}

```

Código-3.32: InternationalStudent.java con un método static

4. Modificar **Main.java** como se muestra en Código-3.33. El cambio tiene como objetivo llamar métodos ocultos y observar el tipo de método estático invocado. El fragmento de código que se requiere añadir se muestra en azul.

```

package mypeopleexample;

public class Main {

    public static void main(String[] args) {

        System.out.println("---- Observe overriding behavior ----");
    }
}

```

```

Person person1 = new Person();
person1.myMethod("test1");

Student student1 = new Student();
student1.myMethod("test2");

InternationalStudent internationalStudent1 =
    new InternationalStudent();
internationalStudent1.myMethod("test3");

// Polymorphic behavior
System.out.println("---- Observe polymorphic behavior ----");
Person person2 = new Student();
person2.myMethod("test4");

Person person3 = new InternationalStudent();
person3.myMethod("test5");

Student student2 = new InternationalStudent();
student2.myMethod("test6");

// Calling hiding methods
System.out.println("---- Observe how calling hiding methods work ----");
person2.myStaticMethod("test7");
person3.myStaticMethod("test8");
student2.myStaticMethod("test9");
}
}

```

Código-3.33: Main.java

5. Compilar y ejecutar el programa.

- Hacer click con el botón derecho en el proyecto **MyPeopleExampleHidingMethods** y seleccionar **Run**.
- Observar el resultado en la ventana **Output** de NetBeans prestando atención a la parte en negrita. (Figura-3.34)

```

---- Observe overriding behavior ----
myMethod(test1) in Person class
myMethod(test2) in Student class
myMethod(test3) in InternationalStudent class
---- Observe polymorphic behavior ----
myMethod(test4) in Student class
myMethod(test5) in InternationalStudent class
myMethod(test6) in InternationalStudent class
---- Observe how calling hiding methods work ----
myStaticMethod(test7) in Person class
myStaticMethod(test8) in Person class
myStaticMethod(test9) in Student class

```

Figura-3.34: Resultado

[Volver al inicio del ejercicio](#)

Ejercicio 4: casting de tipos

En este ejercicio se mostrará el casting de tipos entre tipos de clases que están relacionadas a través de la herencia.

1. **Casting de tipo implícito**
2. **Casting de tipo explícito con un runtime ClassCastException**
3. **Casting de tipo explícito sin un runtime ClassCastException**

(4.1) Casting de tipo implícito

Se puede usar el proyecto **MyPeopleExample** para los cambios que se van a hacer en esta sección o se puede crear un nuevo proyecto, por ejemplo, **MyPeopleExampleImplicitCasting**, "copiando" el proyecto **MyPeopleExample** - Hacer click con el botón derecho en el proyecto **MyPeopleExample** y seleccionar **Copy Project**. En lo que sigue se asume que se ha creado un nuevo proyecto.

1. Modificar the **Main.java** como se muestra en Código-4.11. El cambio consiste en añadir unas cuantas sentencias en las que se realiza el casting de tipo implícito entre tipos.

```

package mypeopleexample;

public class Main {

    public static void main(String[] args) {

        System.out.println("---- Observe overriding behavior ----");
        Person person1 = new Person();
        person1.myMethod("test1");

        Student student1 = new Student();
        student1.myMethod("test2");

        InternationalStudent internationalStudent1 =
            new InternationalStudent();
        internationalStudent1.myMethod("test3");

        // Polymorphic behavior
        System.out.println("---- Observe polymorphic behavior ----");
    }
}

```

```

// This is an implicit type casting between Student and Person class.
Person person2 = new Student(); // Example 1
person2 = student1; // Example 2
person2.myMethod("test4");

// This is an implicit type casting between InternationalStudent and Person class.
Person person3 = new InternationalStudent(); // Example 3
person3 = internationalStudent1; // Example 4
person3.myMethod("test5");

// This is an implicit type casting between InternationalStudent and Student class.
Student student2 = new InternationalStudent(); // Example 5
student2 = internationalStudent1; // Example 6
student2.myMethod("test6");

// Calling hiding methods
System.out.println("---- Observe how calling hiding methods work ----");
person2.myStaticMethod("test7");
person3.myStaticMethod("test8");
student2.myStaticMethod("test9");
}
}

```

Código-4.11: Main.java

2. Compilar y ejecutar el programa

- Hacer click con el botón derecho en el proyecto **MyPeopleExampleImplicitCasting** y seleccionar **Run**.
- Observar el resultado en la ventana Output de NetBeans. (Figura-4.12)

```

---- Observe overriding behavior ----
myMethod(test1) in Person class
myMethod(test2) in Student class
myMethod(test3) in InternationalStudent class
---- Observe polymorphic behavior ----
myMethod(test4) in Student class
myMethod(test5) in InternationalStudent class
myMethod(test6) in InternationalStudent class
---- Observe how calling hiding methods work ----
myStaticMethod(test7) in Person class
myStaticMethod(test8) in Person class
myStaticMethod(test9) in Student class

```

Figura-4.12: Resultado

[Volver al inicio del ejercicio](#)

(4.2) Casting de tipo explícito con un runtime ClassCastException

Se puede usar el proyecto **MyPeopleExample** para los cambios que se van a hacer en esta sección o se puede crear un nuevo proyecto, por ejemplo, **MyTypeMismatchExampleProject1**, "copiando" el proyecto **MyPeopleExample** - Hacer click con el botón derecho en el proyecto **MyPeopleExample** y seleccionar **Copy Project**. En lo que sigue se asume que se ha creado un nuevo proyecto.

1. Modificar the **Main.java** como se muestra en Código-4.21.

```

package mytypemismatchexampleproject;

public class Main {

    public static void main(String[] args) {

        // Implicit casting - Student object instance is
        // type of Person.
        Person person1 = new Student();

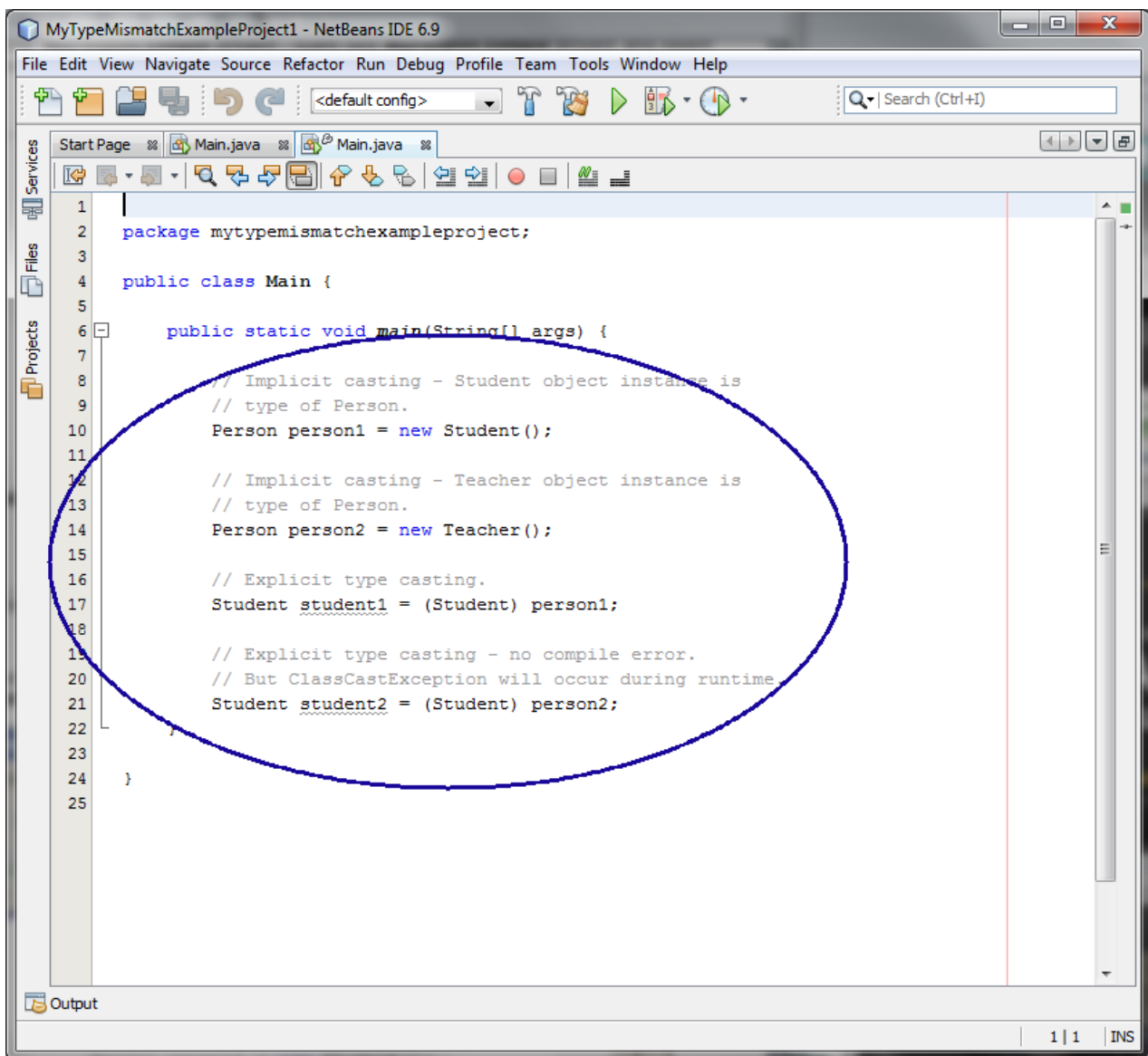
        // Implicit casting - Teacher object instance is
        // type of Person.
        Person person2 = new Teacher();

        // Explicit type casting.
        Student student1 = (Student) person1;

        // Explicit type casting - no compile error.
        // But ClassCastException will occur during runtime.
        Student student2 = (Student) person2;
    }
}

```

Código-4.21: Casting explícito con una exception potencial por mismatch en runtime



2. Compilar y ejecutar el programa

- Hacer click con el botón derecho en el proyecto **MyTypeMismatchExampleProject1** y seleccionar **Run**.
- Observar que se produce una excepción en tiempo de ejecución **java.lang.ClassCastException**.

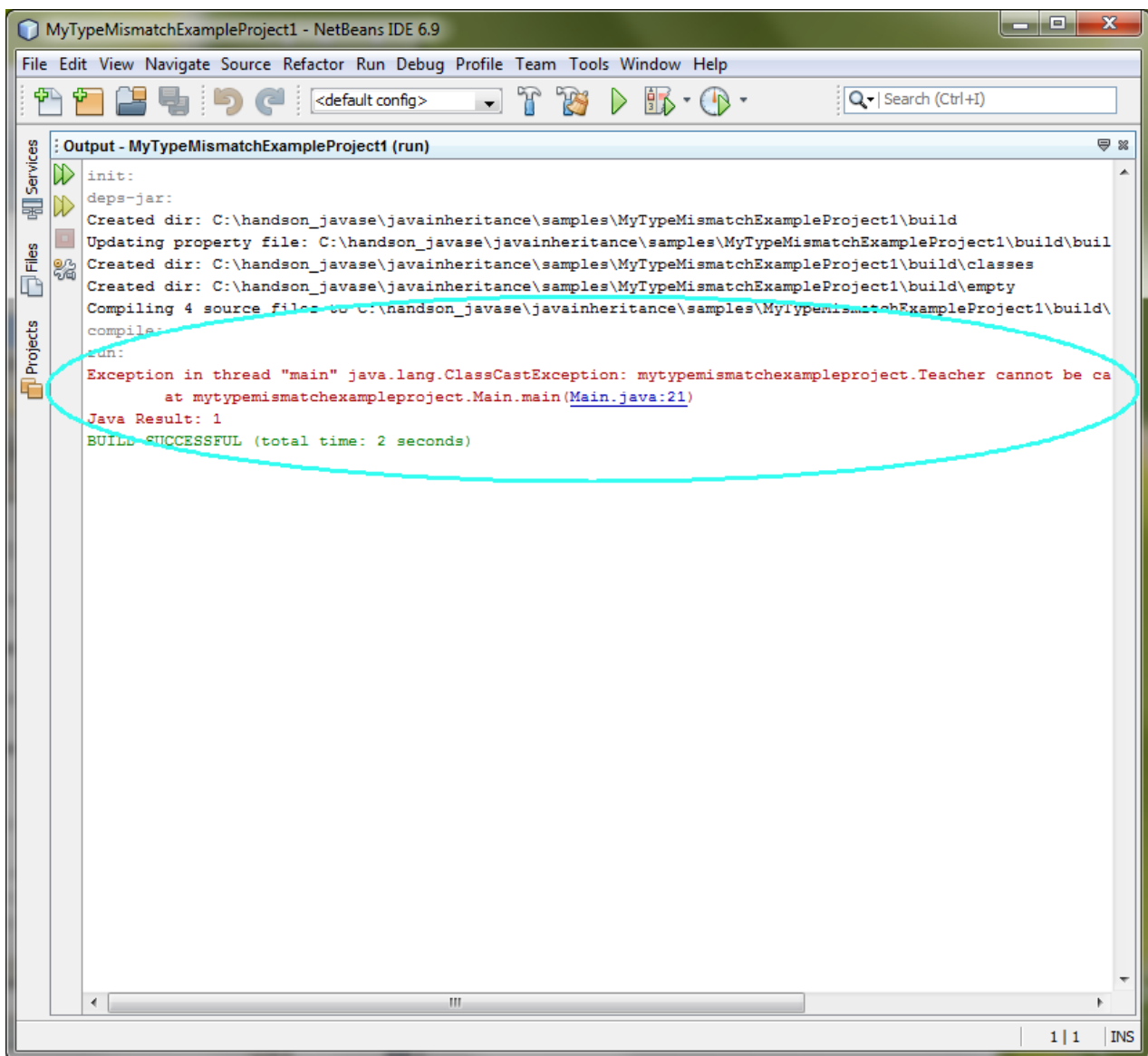


Figura-4.22: ocurrencia de java.lang.ClassCastException

[Volver al inicio del ejercicio](#)

(4.3) Casting de tipo explícito sin un runtime ClassCastException

Se puede usar el proyecto **MyPeopleExample** para los cambios que se van a hacer en esta sección o se puede crear un nuevo proyecto, por ejemplo, **MyTypeMismatchExampleProject2**, "copiando" el proyecto **MyPeopleExample** - Hacer click con el botón derecho en el proyecto **MyPeopleExample** y seleccionar **Copy Project**. En lo que sigue se asume que se ha creado un nuevo proyecto.

1. Modificar **Main.java** como se muestra en Código-4.23. El fragmento de código que se requiere añadir se muestra en azul.

```
package mytypemismatchexampleproject;

public class Main {

    public static void main(String[] args) {

        // Implicit casting - Student object instance is
        // type of Person.
        Person person1 = new Student();

        // Implicit casting - Teacher object instance is
        // type of Person.
        Person person2 = new Teacher();

        // Explicit type casting.
        Student student1 = (Student) person1;

        // Do the casting only when the type is verified
        if (person2 instanceof Student) {
            Student student2 = (Student) person2;
            System.out.println("person2 instanceof Student = " + true);
        } else {
            System.out.println("person2 instanceof Student = " + false);
        }
    }
}
```

```
}
```

Código-4.23: Uso del operador instanceof para comprobar el tipo de la instancia del objeto

2. Compilar y ejecutar el programa.

- Hacer click con el botón derecho en el proyecto **MyTypeMismatchExampleProject2** y seleccionar **Run**.
- Observar el resultado en la ventana Output. (Figura-4.24)

```
person2 instanceof Student = false
```

Figura-4.24: Resultado

[Volver al inicio del ejercicio](#)

Resumen

En este ejercicio se ha visto cómo hacer un casting implícito y explícito entre instancias de objetos que están relacionados a través de la herencia.

[Volver al inicio](#)

Ejercicio 5: Clases Final y métodos Final

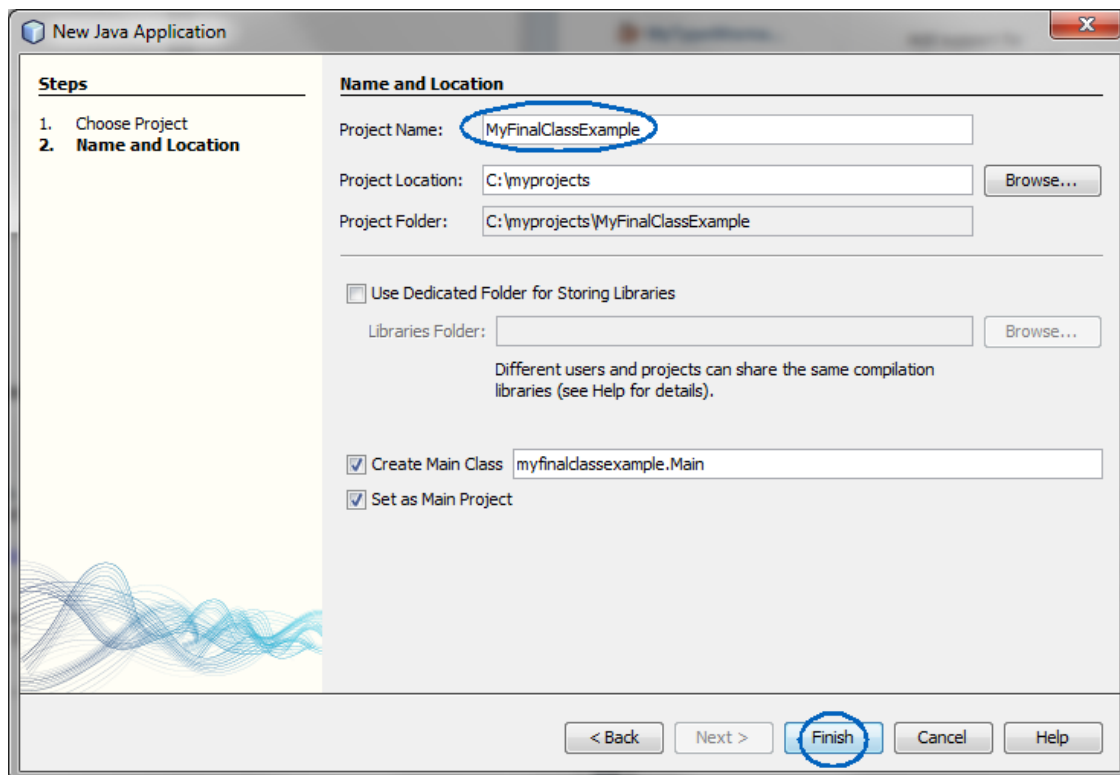
Este ejercicio muestra el concepto de clase final y método final.

1. [Compilar y ejecutar un programa Java con una clase final](#)
2. [Prueba de extender la clase String o la clase Wrapper](#)
3. [Compilar y ejecutar un programa Java con un método final](#)

(5.1) Compilar y ejecutar un programa Java con una clase final

1. Crear un proyecto NetBeans

- Seleccionar **File->New Project (Ctrl+Shift+N)**. El diálogo **New Project** aparece.
- En la sección **Choose Project** seleccionar **Java** en la sección **Categories** y **Java Application** en la sección **Projects**.
- Pulsar en **Next**.
- En el campo **Project Name** escribir **MyFinalClassExample**.
- Click **Finish**.



- Notice that the project node **MyFinalClassExample** is created in the **Projects** section of the NetBeans IDE and that **Main.java** is displayed in the IDE editor window.

2. Write **Person.java** .

- Right click on the **MyFinalClassExample** project and select **New-> Java Class** .
- Notice that the **New Java Class** dialog box appears with the **Name and Location** section .
- In the **Class Name** field write **Person** .
- In the **Package** field , type **myfinalclassexample** or select **myfinalclassexample** from the drop-down menu.
- Click **Finish** .

New Java Class

Steps

1. Choose File Type
2. **Name and Location**

Name and Location

Class Name:

Project:

Location:

Package:

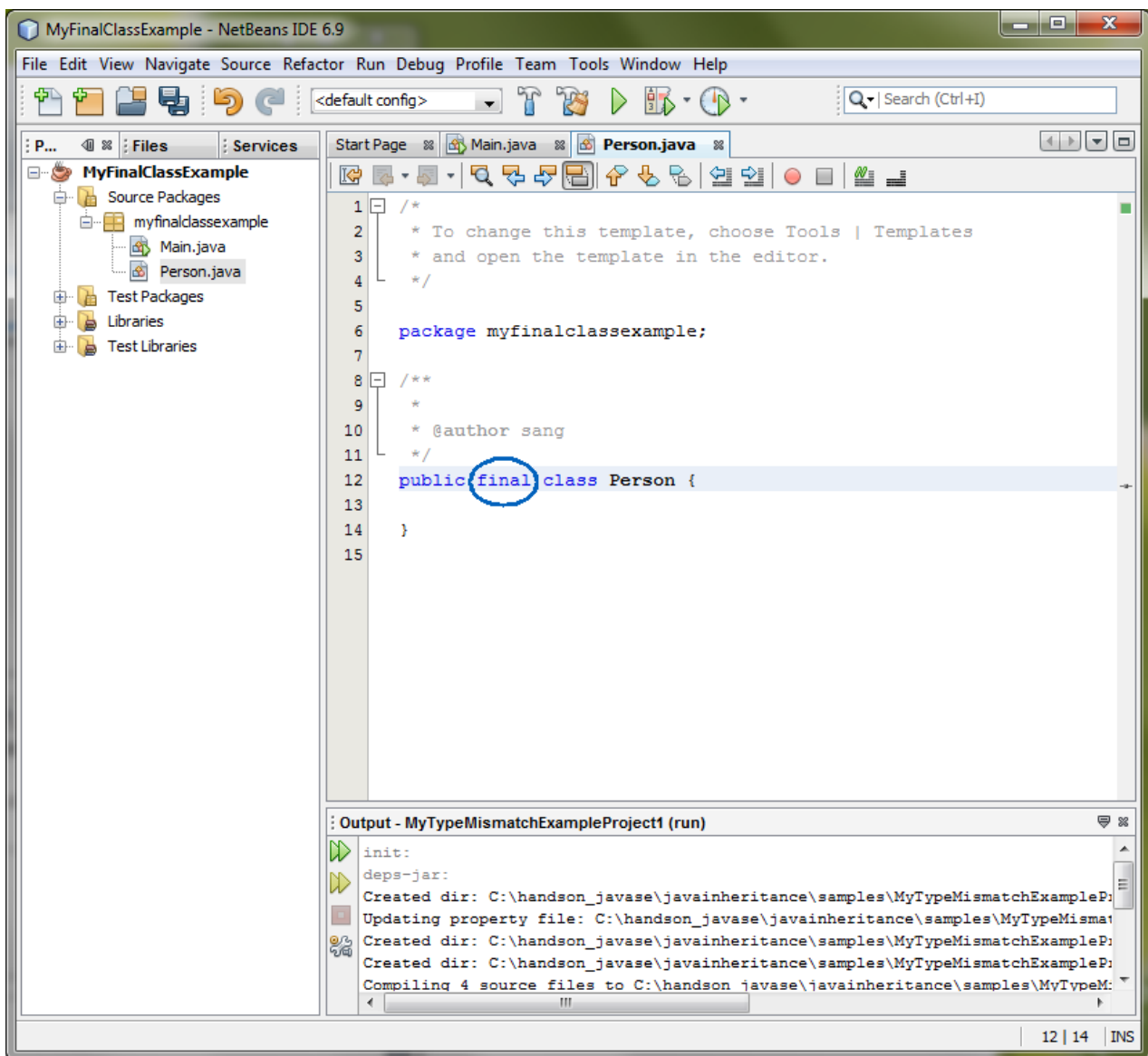
Created File:

< Back Next > **Finish** Cancel Help

- Observe that it is generated and displayed in the editor window **Person.java**.
- Modify **Person.java** as shown in Code-5.10. Note that the **Person** class is a final class, which means that it cannot be extended.

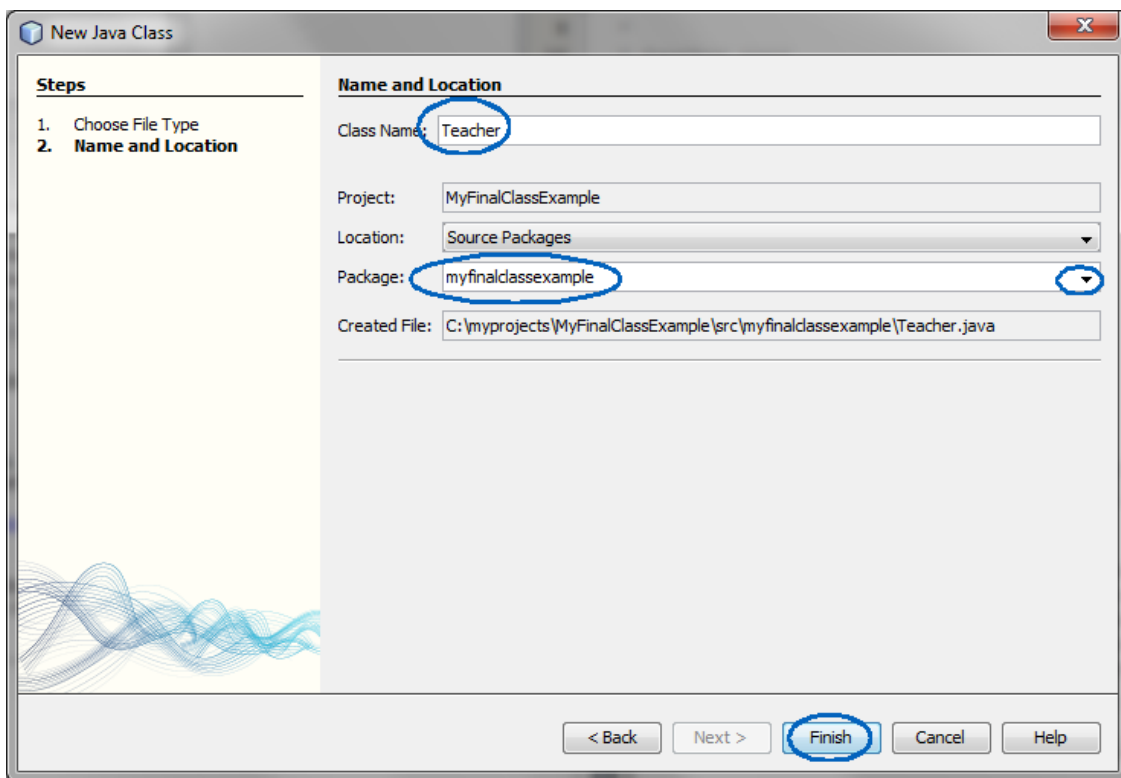
```
package myfinalclassexample;  
  
// Make the Person class as a final class  
public final class Person {  
  
}
```

Code-5.10: Person.java as final class



3. Write **Teacher.java** .

- Right click on the **MyFinalClassExample** project and select **New-> Java Class** .
- Observar que aparece el cuadro de diálogo **New Java Class** con la sección **Name and Location**.
- En el campo **Class Name** escribir **Teacher**.
- En el campo **Package**, escribir **myfinalclassexample** o seleccionar **myfinalclassexample** a partir del menú desplegable .
- Click **Finish**.



- Observar que se genera y muestra en la ventana del editor **Teacher.java**.
- Modificar **Teacher.java** como se muestra en Código-5.11. El cambio extiende la clase **Person** class, lo que producirá un error de compilación.

```
package myfinalclassexample;  
  
/ **  
 *  
 * @author sang  
 */  
public class Teacher extends Person {  
  
}
```

Code-5.11: Teacher.java

4. Observe the compilation error. (Figure-5.12)

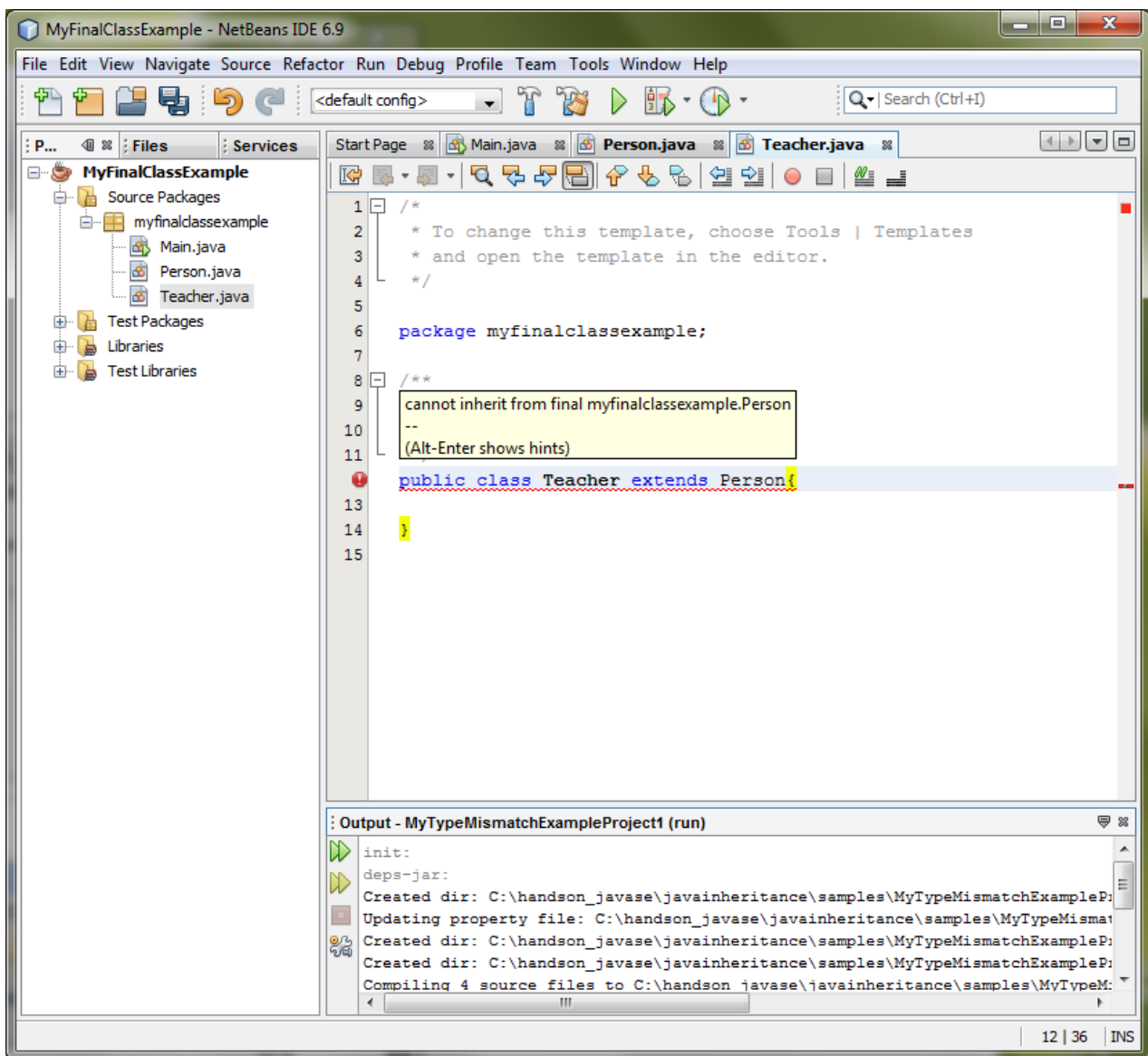


Figure-5.12: Compilation error

[Return to the beginning of the exercise](#)

(5.2) Test to extend String class or Wrapper class

1. Create a NetBeans project

- Select **File -> New Project (Ctrl + Shift + N)** . The **New Project** dialog appears.
- In the **Choose Project** section select **Java** in the **Categories** section and **Java Application** in the **Projects** section .
- Click on **Next** .
- Notice that the **Name and Location** section appears .
- In the **Project Name** field write **MyFinalClassExample2** .
- Click **Finish** .

New Java Application

Steps

1. Choose Project
2. Name and Location

Name and Location

Project Name: MyFinalClassExample2

Project Location: C:\myprojects Browse...

Project Folder: C:\myprojects\MyFinalClassExample2

☐ Use Dedicated Folder for Storing Libraries

Libraries Folder: Browse...

Different users and projects can share the same compilation libraries (see Help for details).

☒ Create Main Class myfinalclassexample2.Main

☒ Set as Main Project

< Back Next > **Finish** Cancel Help

- Observar que aparece el nodo del proyecto **MyFinalClassExample2** se crea en la sección **Projects** del IDE NetBeans y que se muestra **Main.java** en la ventana del editor del IDE.

2. Escribir **Teacher.java**.

- Hacer click con el botón derecho en el proyecto **MyFinalClassExample** y seleccionar **New->Java Class**.
- Observar que aparece el cuadro de diálogo **New Java Class** con la sección **Name and Location**.
- En el campo **Class Name** escribir **Teacher**.
- En el campo **Package**, escribir **myfinalclassexample2** o seleccionar **myfinalclassexample2** a partir del menú desplegable .
- Click **Finish**.

New Java Class

Steps

1. Choose File Type
2. Name and Location

Name and Location

Class Name: Teacher

Project: MyFinalClassExample2

Location: Source Packages

Package: myfinalclassexample2

Created File: C:\myprojects\MyFinalClassExample2\src\myfinalclassexample2\Teacher.java

< Back Next > **Finish** Cancel Help

- Observar que se genera y muestra en la ventana del editor **Teacher.java**.
- Modificar **Teacher.java** como se muestra en Código-5.21. El cambio es para extender la clase String class, lo que produce un error de compilación.

```
package myfinalclassexample2;

/**
 *
 * @author
 */
public class Teacher extends String{
```

}

Código-5.21: Teacher.java

3. Observe the compilation error.

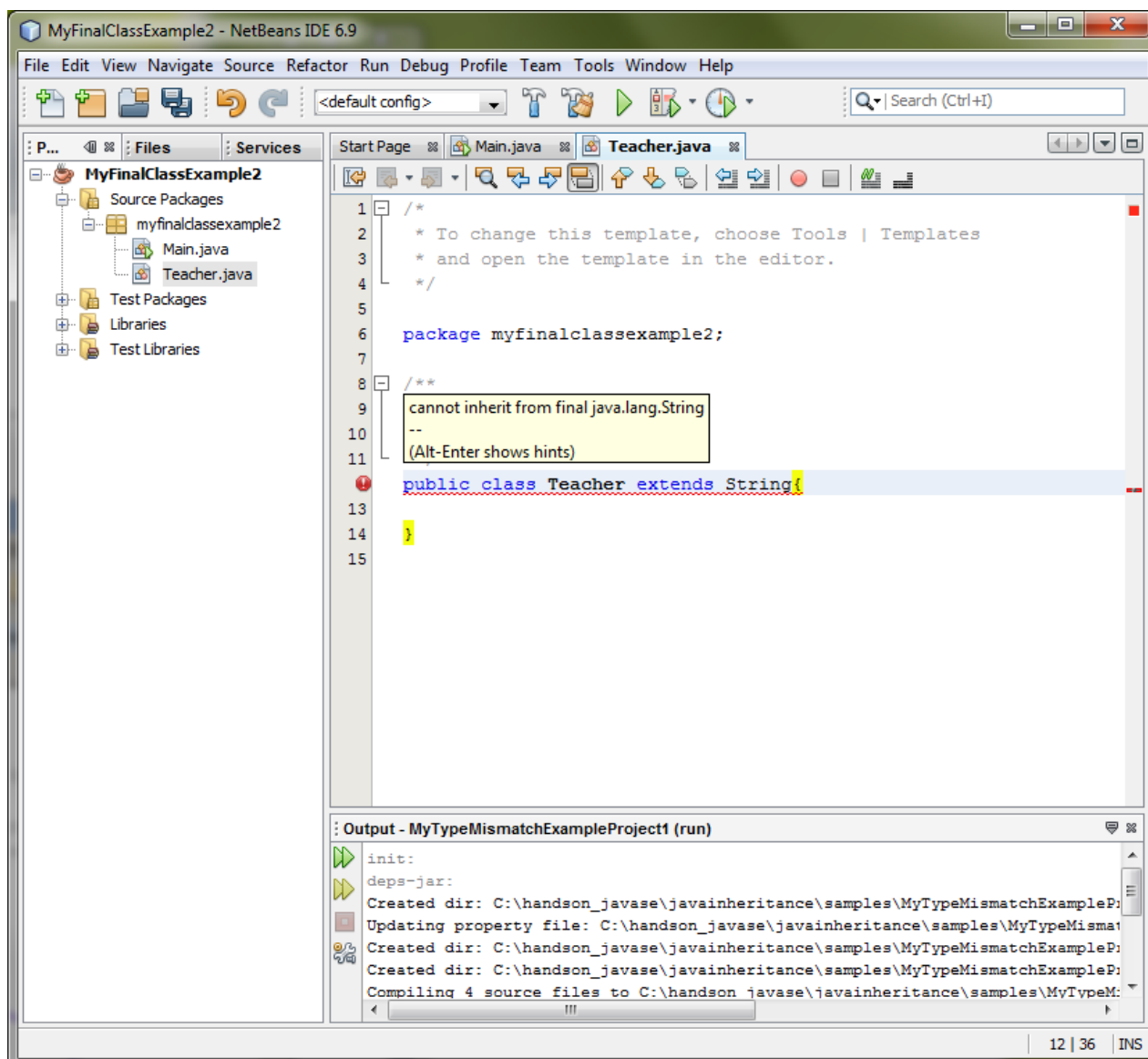
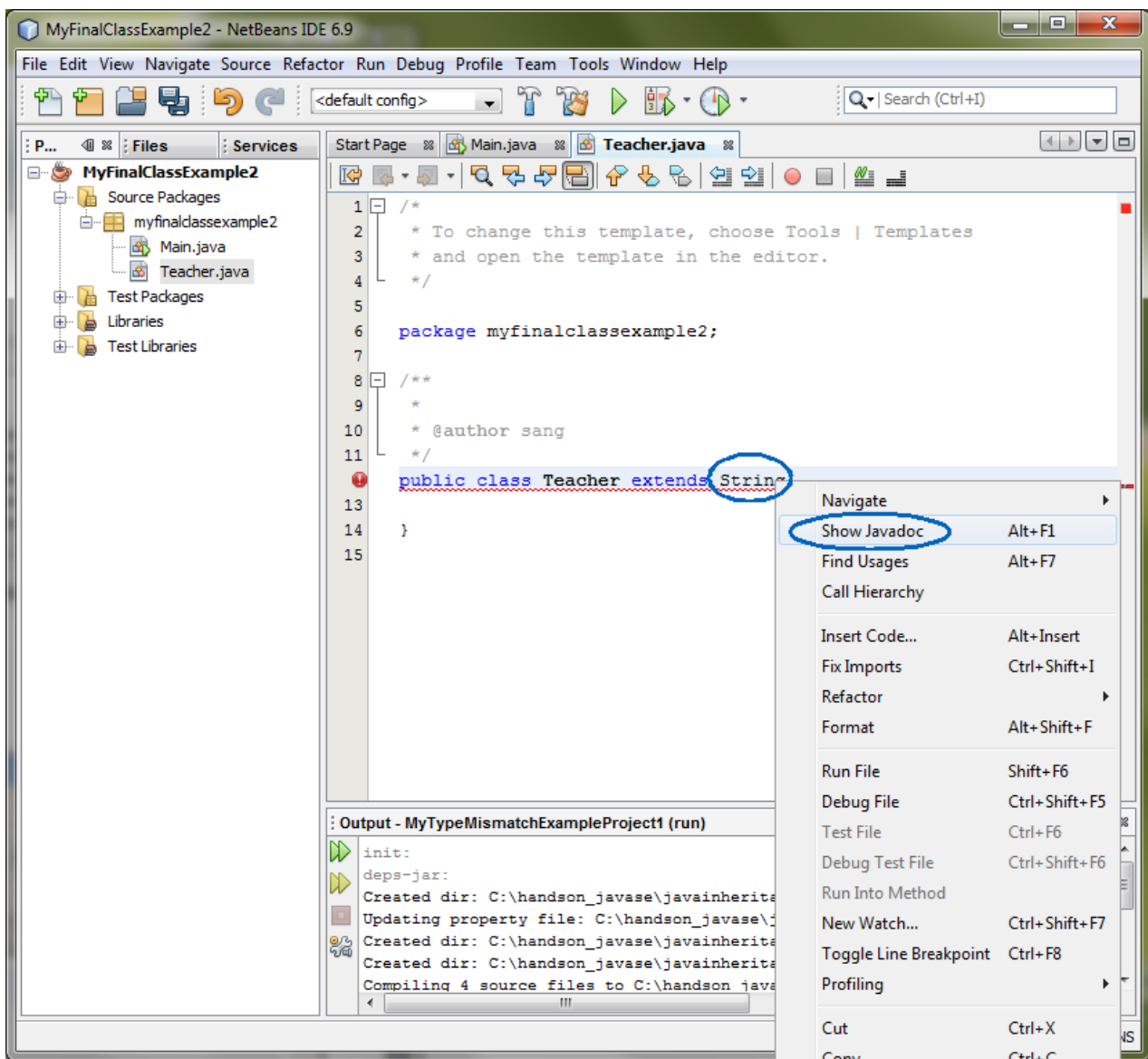


Figure-5.22: Compilation error

4. Show String class Javadoc to check that String class is a final class.

- Move the cursor to **String** and with the right button select **Show Javadoc**.



- Notice that the Javadoc of the **String** class is displayed .
- Notice that the String class is a final class. (Figure-5.23)

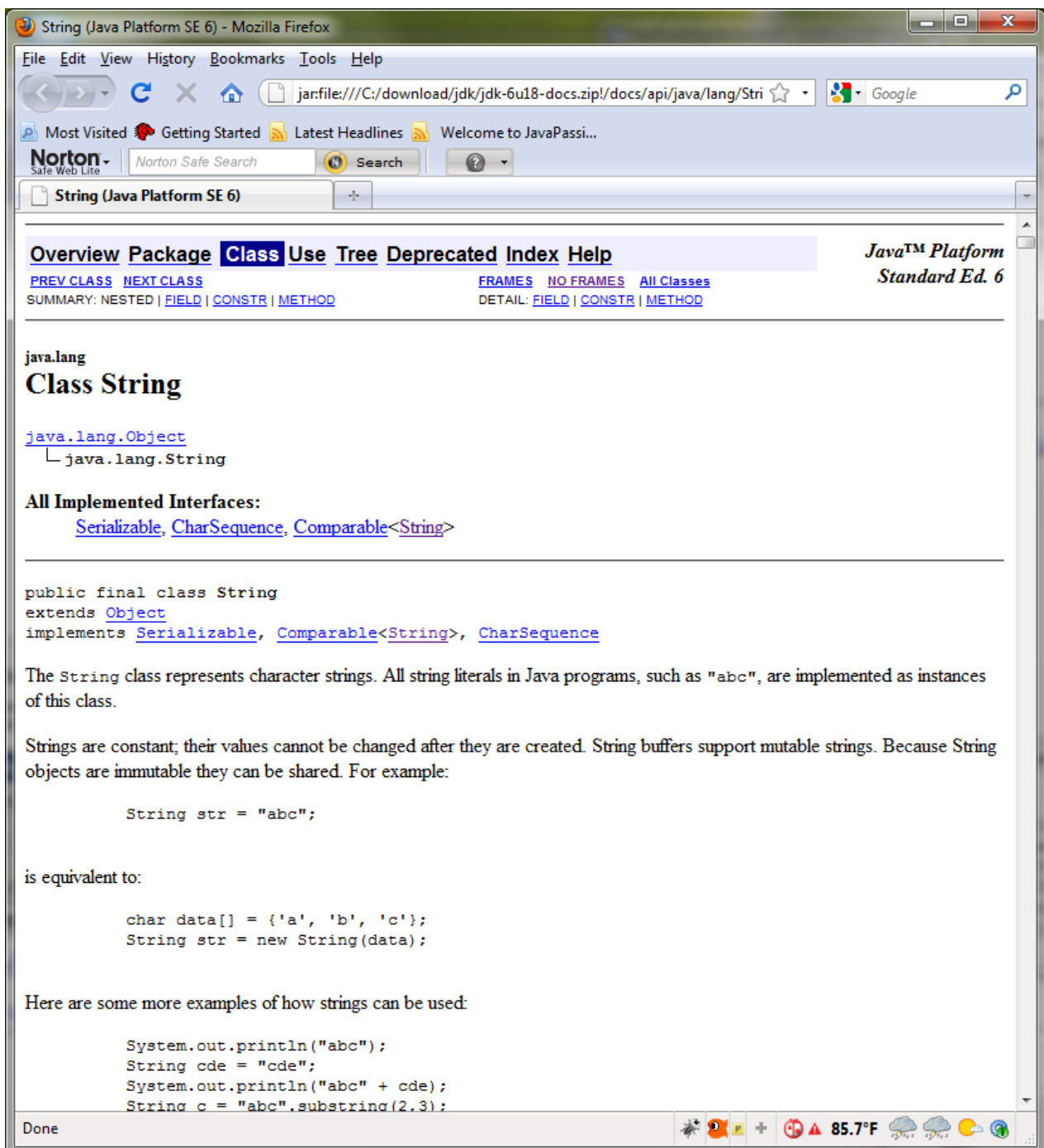


Figure-5.23: Javadoc of the String class, indicates that it is a final class

t.

[Return to the beginning of the exercise](#)

(5.3) Compile and run a Java program with a final method

1. Create a NetBeans project

- Select **File -> New Project (Ctrl + Shift + N)**. The **New Project** dialog appears.
- In the **Choose Project** section select **Java** in the **Categories** section and **Java Application** in the **Projects** section.
- Click on **Next**.
- In the **Name and Location** section, for the **Project Name** field, type **MyFinalMethodExample**.
- Click **Finish**.
- Notice that the project node **MyFinalMethodExample** is created in the **Projects** section of the NetBeans IDE and that **Main.java** is displayed in the IDE editor window.

2. Write **Person.java**.

- Right click on the **MyFinalMethodExample** project and select **New-> Java Class**.
- Notice that the **New Java Class** dialog box appears with the **Name and Location** section.
- In the **Class Name** field write **Person**.
- In the **Package** field, type **myfinalmethodexample** or select **myfinalmethodexample** from the drop-down menu.
- Click **Finish**.

New Java Class

Steps

1. Choose File Type
2. **Name and Location**

Name and Location

Class Name: Person

Project: MyFinalMethodExample

Location: Source Packages

Package: myfinalmethodexample

Created File: C:\myprojects\MyFinalMethodExample\src\myfinalmethodexample\Person.java

< Back Next > Finish Cancel Help

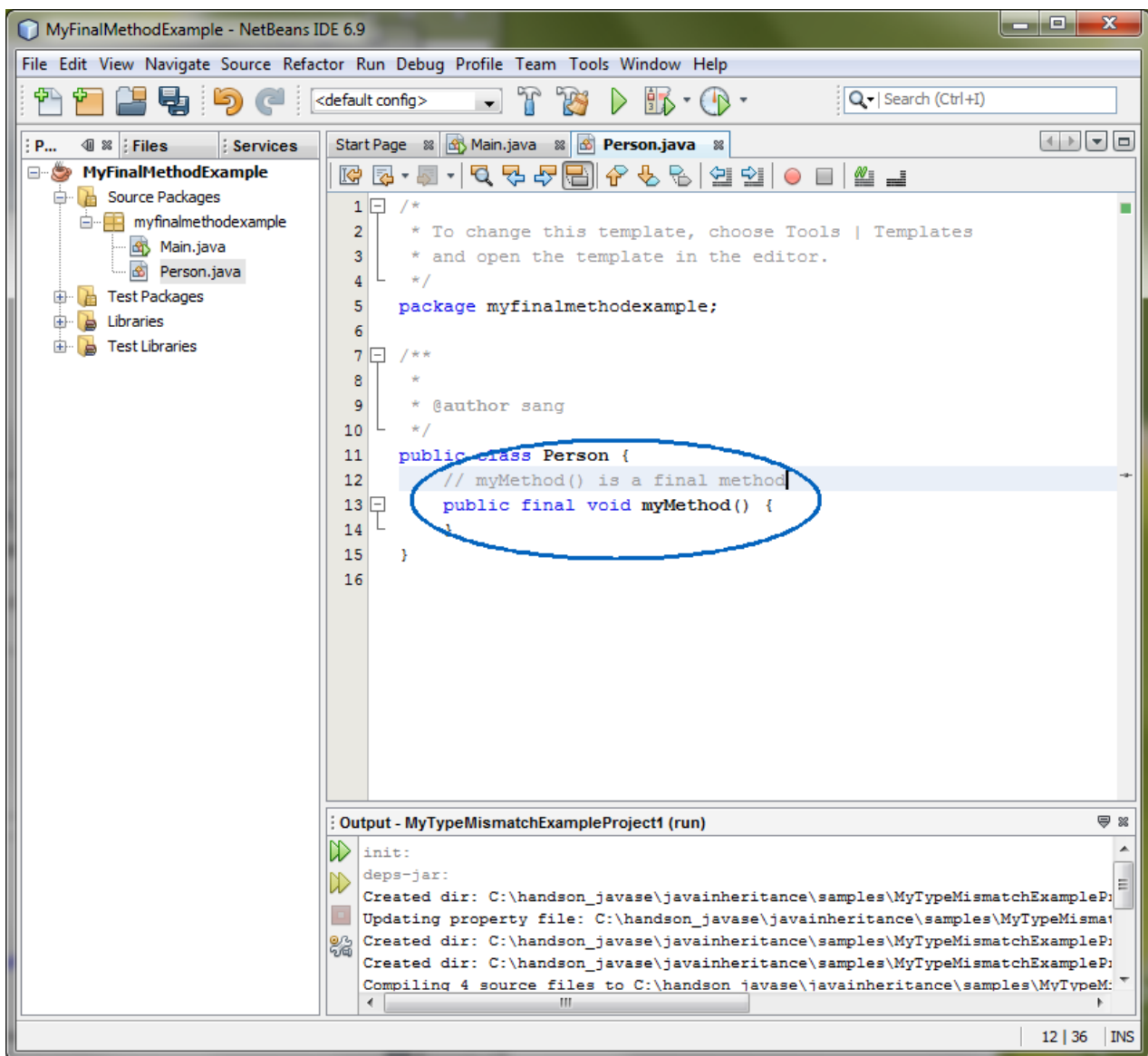
- Observe that it is generated and displayed in the editor window **Person.java** .
- Modify **Person.java** as shown in Code-5.30. Note that the **myMethod** method is a final method, which means that it cannot be used (override) in a subclass.

```
package myfinalmethodexample;

public class Person {

    // myMethod () is a final method
    public final void myMethod () {
    }
}
```

Code-5.30: Person.java has a final method



3. Write **Teacher.java** .

- Hacer click con el botón derecho en el proyecto **MyFinalMethodExample** y seleccionar **New->Java Class**.
- Observar que aparece el cuadro de diálogo **New Java Class** con la sección **Name and Location**.
- En el campo **Class Name** escribir **Teacher**.
- En el campo **Package**, escribir **myfinalmethodexample** o seleccionar **myfinalmethodexample** a partir del menú desplegable .
- Click **Finish**.
- Observar que se genera y muestra en la ventana del editor **Teacher.java**.
- Modify **Teacher.java** as shown in Code-5.31. The change is for the Teacher class to derive from the Person class and override the **myMethod ()** of the **Person** class , which produces a compilation error.

```

package myfinalmethodexample;

public class Teacher extends Person {

    // Try to override this method
    public void myMethod () {
    }

}

```

Code-5.31: Teacher.java

4. Observe the compilation error. (Figure-5.32)

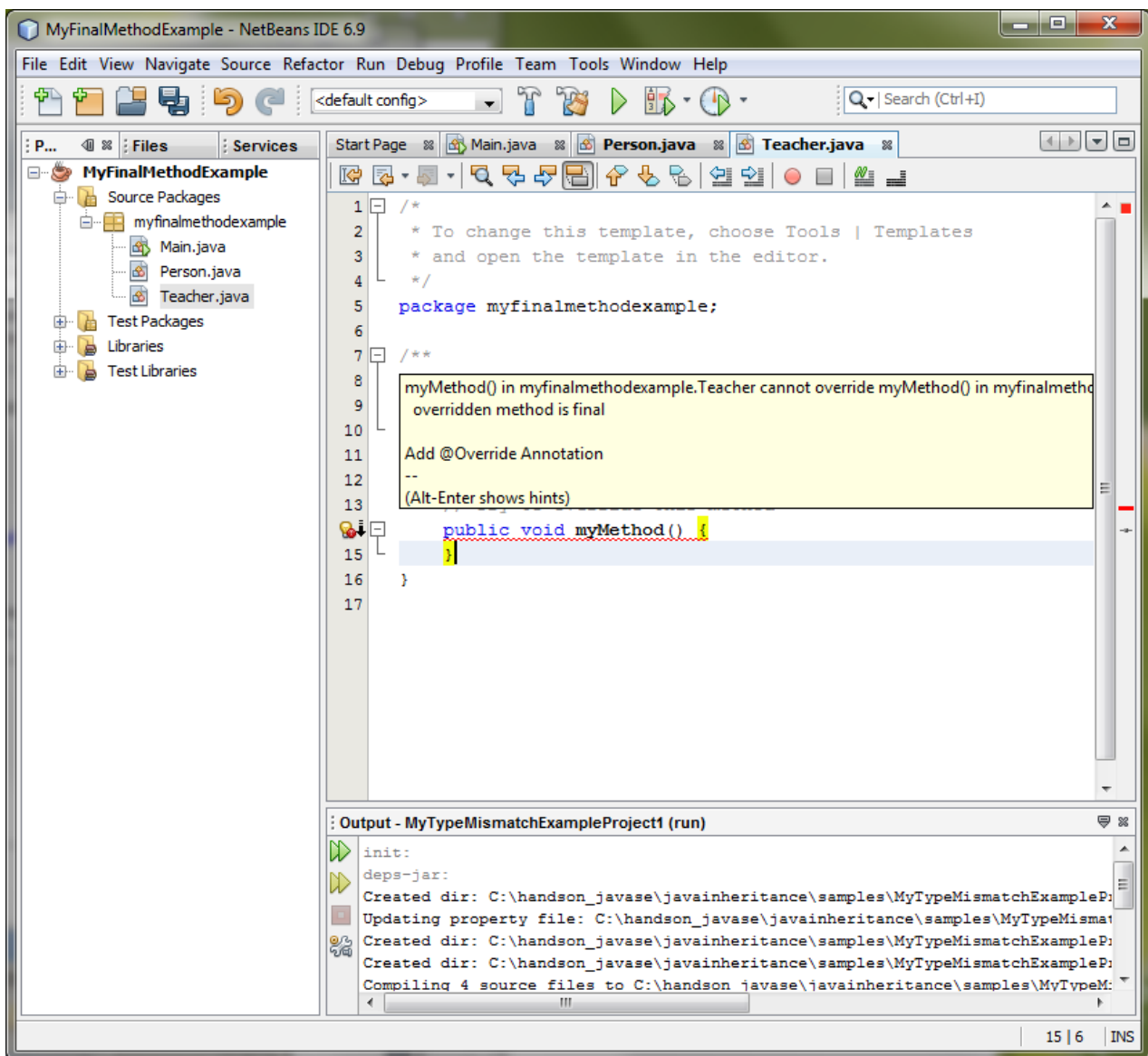


Figure-5.32: Compilation error

[Return to the beginning of the exercise](#)

Summary

In this exercise we have seen that a final class cannot be derived and that a final method cannot be overridden by a subclass b.

[Back to top](#)

Exercise 6: Developing a simple program using inheritance

In this exercise, you develop a simple program using several classes that are related by inheritance. The Product class is inherited from the Electronics and Book classes. The Electronics class is derived from the MP3Player and TV classes. You can also see how to add polymorphic behavior to the program through method overriding.

1. Build the MyOnlineShop program

(6.1) Build the MyOnlineShop program

1. Create a NetBeans project

- Select **File -> New Project (Ctrl + Shift + N)** . The **New Project** dialog appears.
- In the **Choose Project** section select **Java** in the **Categories** section and **Java Application** in the **Projects** section .
- Click on **Next** .
- Notice that the **Name and Location** pane appears.
- For the **Project Name** field, enter **MyOnlineShop** .
- Click on **Finish** .

2. Write **Product.java** .

- Right click on the **MyOnlineShop** project and select **New-> Java Class** .
- Notice that the **New Java Class** dialog box appears with the **Name and Location** section .
- In the **Class Name** field write **Product** .
- In the **Package** field , type **myonlineshop** or select **myonlineshop** from the dropdown menu.
- Click on **Finish** .
- Observe that it is generated and displayed in the **Product.java** editor window .
- Modify **Product.java** as shown in Code-6.10.

```

package myonlineshop;

public class Product {

    private double regularPrice;

    /** Creates a new instance of Product */
    public Product (double regularPrice) {
        this.regularPrice = regularPrice;
    }

    // Method that will be overridden
    public double computeSalePrice () {
        return 0;
    }

    public double getRegularPrice () {
        return regularPrice;
    }

    public void setRegularPrice (double regularPrice) {
        this.regularPrice = regularPrice;
    }

}

```

Code-6.10: Product.java

3. Write **Electronics.java** .

- Right click on the **MyFinalClassExample** project and select **New-> Java Class** .
- Notice that the **New Java Class** dialog box appears with the **Name and Location** section .
- In the **Class Name** field write **Electronics** .
- In the **Package** field , type **myonlineshop** or select **myonlineshop** from the drop-down menu.
- Click **Finish** .
- Observe that it is generated and displayed in the editor window **Electronics.java** .
- Modify **Electronics.java** as shown in Code-6.11. Note that the Electronics class extends the Product class.

```

package myonlineshop;

public class Electronics extends Product {

    private String manufacturer;

    /** Creates a new instance of Electronics */
    public Electronics (double regularPrice,
        String manufacturer) {
        super (regularPrice);
        this.manufacturer = manufacturer;
    }

    // Override this method
    public double computeSalePrice () {
        return super.getRegularPrice () * 0.6;
    }

    public String getManufacturer () {
        return manufacturer;
    }

    public void setManufacturer (String manufacturer) {
        this.manufacturer = manufacturer;
    }

}

```

Code-6.11: Electronics.java

4. Write **MP3Player.java** .

- Right click on the **MyFinalClassExample** project and select **New-> Java Class** .
- Notice that the **New Java Class** dialog box appears with the **Name and Location** section .
- In the **Class Name** field write **MP3Player** .
- In the **Package** field , type **myonlineshop** or select **myonlineshop** from the drop-down menu.
- Click **Finish** .
- Observe that it is generated and displayed in the **MP3Player.java** editor window .
- Modify **MP3Player.java** as shown in Code-6.12. Note that **MP3Player** class extends **Electronics** class. Note that the **MP3Player** class has a **computeSalePrice ()** method which is an overriding method.

```

package myonlineshop;

public class MP3Player extends Electronics {

    private String color;

    /**
     * Creates a new instance of MP3Player
     */
    public MP3Player (double regularPrice,
        String manufacturer,

```

```

        String color) {
    super (regularPrice, manufacturer);
    this.color = color;
}

// Override this method
public double computeSalePrice () {
    return super.getRegularPrice () * 0.9;
}

public String getColor () {
    return color;
}

public void setColor (String color) {
    this.color = color;
}
}

```

Code-6.12: MP3Player.java

5. Write **TV.java** .

- Right click on the **MyFinalClassExample** project and select **New-> Java Class** .
- Notice that the **New Java Class** dialog box appears with the **Name and Location** section .
- In the **Class Name** field write **TV** .
- In the **Package** field , type **myonlineshop** or select **myonlineshop** from the drop-down menu.
- Click **Finish** .
- Observe that it is generated and displayed in the editor window **TV.java** .
- Modify **TV.java** as shown in Code-6.14. Note that the **TV** class extends the **Electronics** class . Also note that the **TV** class has a **computeSalePrice ()** method which is an overriding method.

```

package myonlineshop;

public class TV extends Electronics {

    int size;

    /** Creates a new instance of TV */
    public TV (double regularPrice,
        String manufacturer,
        int size) {
        super (regularPrice, manufacturer);
        this.size = size;
    }

    // Override this method
    public double computeSalePrice () {
        return super.getRegularPrice () * 0.8;
    }
}

```

Code-6.14: TV.java

6. Write **Book.java** .

- Right click on the **MyFinalClassExample** project and select **New-> Java Class** .
- Notice that the **New Java Class** dialog box appears with the **Name and Location** section .
- In the **Class Name** field write **Book** .
- In the **Package** field , type **myonlineshop** or select **myonlineshop** from the drop-down menu.
- Click **Finish** .
- Observe that it is generated and displayed in the **Book.java** editor window .
- Modify **Book.java** as shown in Code-6.15. Notice that the **Book** class extends the **Product** class . Note that the **Book** class also has the **computeSalePrice ()** method which is an overriding method.

```

package myonlineshop;

public class Book extends Product {

    private String publisher;
    private int yearPublished;

    /** Creates a new instance of Book */
    public Book (double regularPrice,
        String publisher,
        int yearPublished) {
        super (regularPrice);
        this.publisher = publisher;
        this.yearPublished = yearPublished;
    }

    // Override this method
    public double computeSalePrice () {
        return super.getRegularPrice () * 0.5;
    }

    public String getPublisher () {
        return publisher;
    }

    public void setPublisher (String publisher) {

```

```

        this.publisher = publisher;
    }

    public int getYearPublished () {
        return yearPublished;
    }

    public void setYearPublished (int yearPublished) {
        this.yearPublished = yearPublished;
    }
}

```

Code-6.15: Book.java

7. Modify **Main.java** as shown in Code-6.16. Study the code paying attention to the comments in bold:

```

package myonlineshop;

public class Main {

    public static void main (String [] args) {

        // Declare and create Product array of size 5
        Product [] pa = new Product [5];

        // Create object instances
        pa [0] = new TV (1000, "Samsung", 30);
        pa [1] = new TV (2000, "Sony", 50);
        pa [2] = new MP3Player (250, "Apple", "blue");
        pa [3] = new Book (34, "Sun press", 1992);
        pa [4] = new Book (15, "Korea press", 1986);

        // Compute total regular price and total
        // sale price.
        double totalRegularPrice = 0;
        double totalSalePrice = 0;

        for (int i = 0; i < pa.length; i++) {

            // Call a method of the super class to get
            // the regular price.
            totalRegularPrice += pa [i] .getRegularPrice ();

            // Since the sale price is computed differently
            // depending on the product type, overriding
            // method of the object instance of the sub-class
            // gets invoked. This is runtime polymorphic
            // behavior.
            totalSalePrice += pa [i] .computeSalePrice ();

            System.out.println ("Item number" + i +
                ": Type =" + pa [i] .getClass (). GetName () +
                ", Regular price =" + pa [i] .getRegularPrice () +
                ", Sale price =" + pa [i] .computeSalePrice () );
        }
        System.out.println ("totalRegularPrice =" + totalRegularPrice);
        System.out.println ("totalSalePrice =" + totalSalePrice);
    }
}

```

Code-6.16: Main.java

8. Compile and run the program.

- Right click on the **MyOnlineShop** project and select **Run** .
- Observe the result in the Output window. (Figure-6.17)

```

Item number 0: Type = myonlineshop.TV, Regular price = 1000.0, Sale price = 800.0
Item number 1: Type = myonlineshop.TV, Regular price = 2000.0, Sale price = 1600.0
Item number 2: Type = myonlineshop.MP3Player, Regular price = 250.0, Sale price = 225.0
Item number 3: Type = myonlineshop.Book, Regular price = 34.0, Sale price = 17.0
Item number 4: Type = myonlineshop.Book, Regular price = 15.0, Sale price = 7.5
totalRegularPrice = 3299.0
totalSalePrice = 2649.5

```

Figure-6.17: Result

9. As an exercise, modify **MyOnlineShop** as follows:

- Add the **Camera** class as a subclass of the **Electronics** class
- Calculate the sale price of Camera with the following business logic
 - Regular price * 0.7
- In **Main.java** , initialize two objects of class Camera

10. As an exercise, modify **MyOnlineShop** as follows:

- Add another overriding method to the classes as follows
 - double computeSpecialCustomerPrice ()
 - The calculation logic should be as follows:
 - For TV, subtract 100 from the sale price
 - For MP3Player, subtract 15 from the sale price

- For Book, subtract 2 from the sale price
- In Main.java, show Special Customer Price for each item of products

Summary

In this exercise, a simple program has been developed that uses several classes that are related by inheritance. Polymorphic behavior has been tested through overloaded methods.

[Back to top](#)

Homework

1. Create a NetBeans project called " **MyOwnAutoShopProject** " as follows:
 - Create a superclass called Car. The Car class has the following fields and methods.
 - int speed;
 - double regularPrice;
 - String color;
 - double getSalePrice ();
 - Create a subclass of Car and name it Truck. The Truck class has the following fields and methods.
 - int weight;
 - double getSalePrice (); // If weight> 2000, 10% discount. Otherwise, 20% discount.
 - Create a subclass of Car and name it Ford. The Ford class has the following fields and methods
 - int year;
 - int manufacturerDiscount;
 - double getSalePrice (); // From the calculated sale price of the Car class, subtract manufacturerDiscount.
 - Create a subclass of Car and name it Sedan. The Sedan class has the following fields and methods.
 - int length;
 - double getSalePrice (); // If length> 20 feet, 5% discount, Otherwise, 10% discount.
 - Create the MyOwnAutoShop class that contains the main () method. Do the following inside the main () method.
 - Create an object of class Sedan and initialize all fields with appropriate values. Use the super (...) method in the constructor to initialize the fields of the super class.
 - Create two objects of class Ford and initialize all fields with appropriate values. Use super (...) in the constructor to initialize the fields of the super class.
 - Create an object of class Car and initialize all fields with appropriate values.
 - Show sale prices for all instances.